算法设计与分析（第六讲）

黄 橙

计算与智能创新学院

chuang@fudan.edu.cn

课程总览

判断一个算法好不好

正确性，复杂度，均摊分析

介绍基本算法思想

分治，随机，数据流算法

学习经典算法范式

动态规划，贪心，回溯

求解网络和路径类问题

最短路径，网络流，二分匹配

求解复杂算法问题

线性规划，在线算法，AI算法

理解高效计算的边界

P vs NP，NP 完全性，近似算法

假设你在玩一个游戏

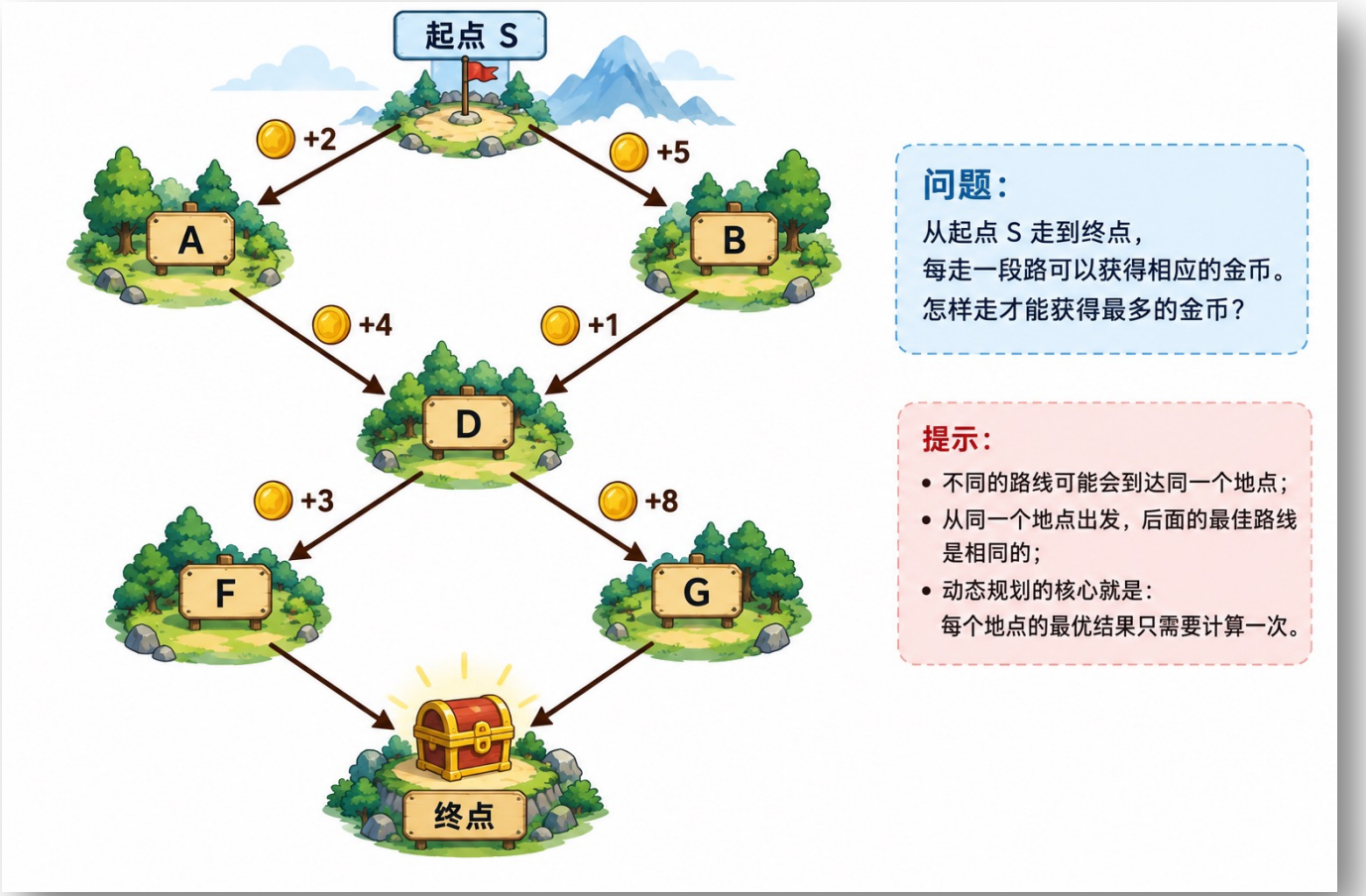
- 假设你在玩一个游戏。每走一步，都有几个选择。你想找到一条收益最高的路线。
- **最简单的方法是什么？**



动态规划（Dynamic Programming）

- **如果把所有路线都试一遍，当然能找到收益最大的路线。但路线可能指数级多。**
- **动态规划要研究的是：**面对大量可能的选择，如何避免把所有路径都试一遍，而是抓住其中重复的状态，用更少的计算找到最优解。

吃金币问题



- ✓ $S \to A \to D \to F$：$2+4+3=9$
- ✓ $S \to A \to D \to G$：$2+4+8=14$
- ✓ $S \to B \to D \to F$：$5+1+3=9$
- ✓ $S \to B \to D \to G$：$5+1+8=14$

从 A 到 D 之后，要比较一次 F 和 G

从 B 到 D 之后，又把 F 和 G 比较了一遍

吃金币问题

- **其实到了 D 以后，前面怎么来的已经不重要了。**
- 从 D 出发
  - 要么，D → F：+3
  - 要么，D → G：+8
- 所以只需要算一次：从 D 到终点的最大收益 = **8**；以后不管是从 A 来到 D，还是从 B 来到 D，直接用这个结果。

动态规划的关键不是把每条完整路线都试一遍，而是：遇到同一个局面时，后面的最优结果只算一次。



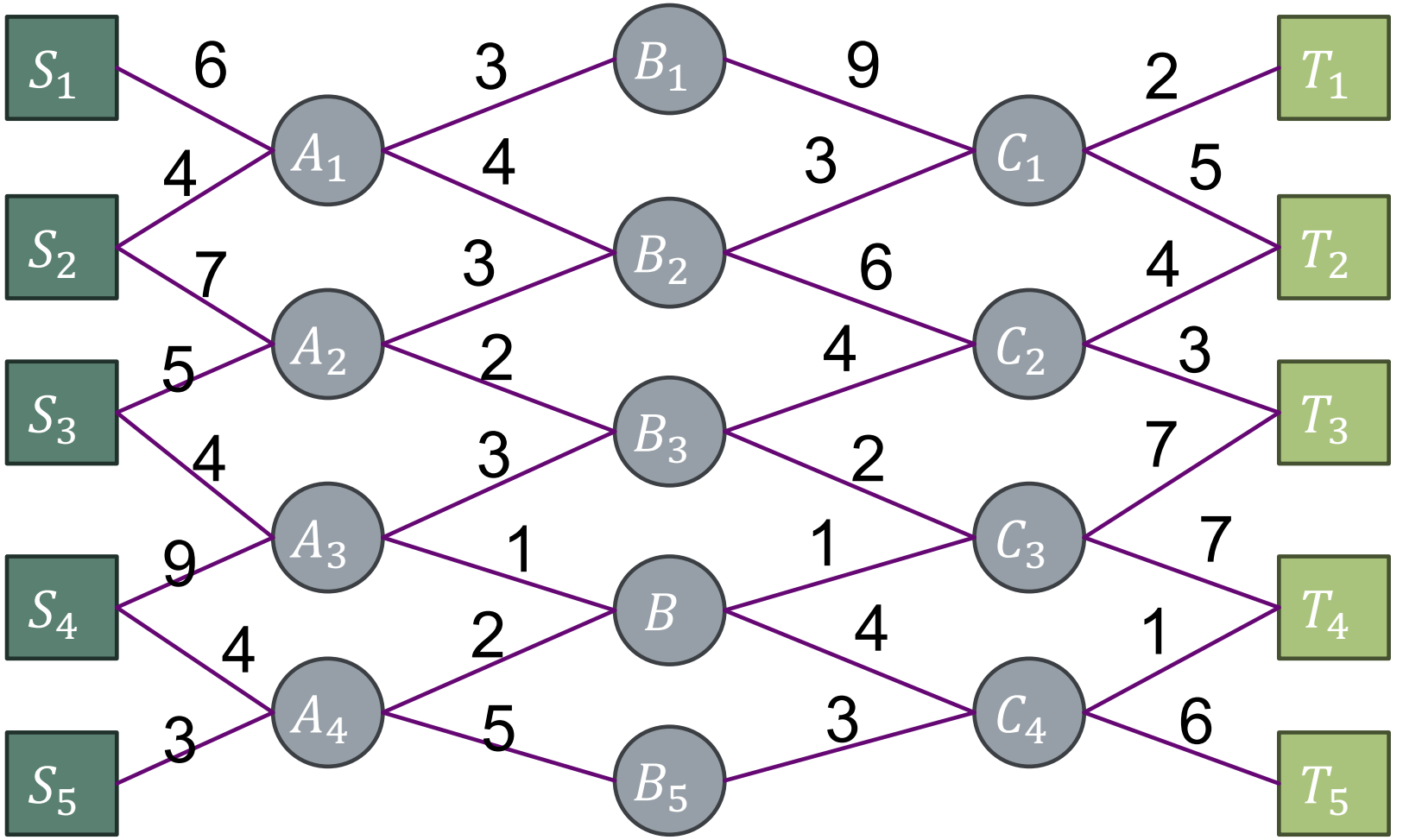
不同的路径，可能会到达同一个状态。同一个状态的最优解，只需要计算一次。

最短路径问题

- 输入
  - 起点集合$\{S_1, S_2, \dots, S_n\}$
  - 终点集合$\{T_1, T_2, \dots, T_n\}$
  - 中间结点集合
  - 边集E，对于任意边 e 有长度

- 输出：一条从起点到终点的最短路径

例子

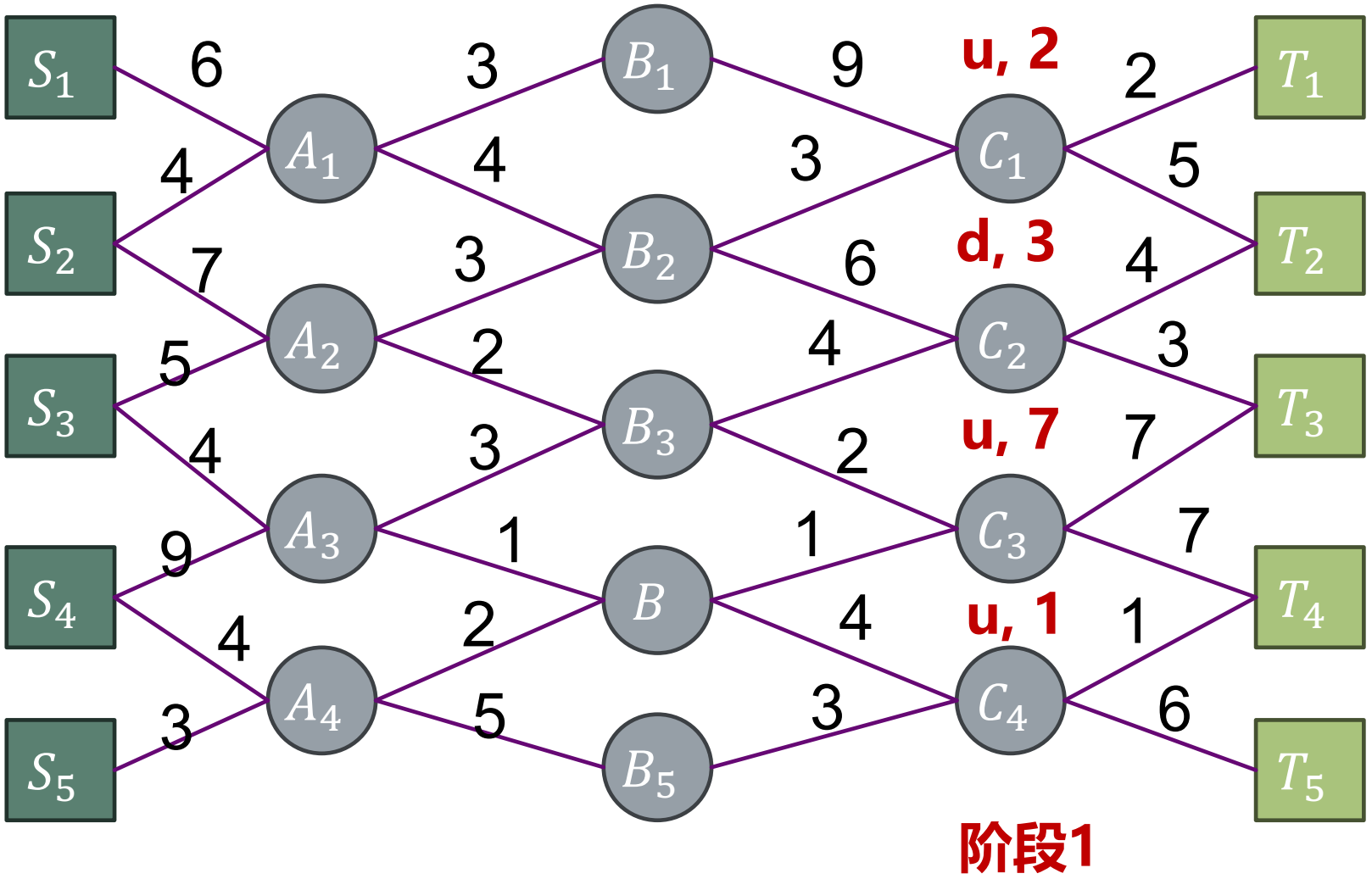

S_1
S_2
S_3
S_4
S_5
A_1
A_2
A_3
A_4
B_1
B_2
B_3
B
B_5
C_1
C_2
C_3
C_4
T_1
T_2
T_3
T_4
T_5
6
4
7
5
4
9
4
3
3
4
3
2
3
2
1
2
5
9
3
4
2
1
4
3
2
1
4
7
5
3
7
7
1
6

算法设计

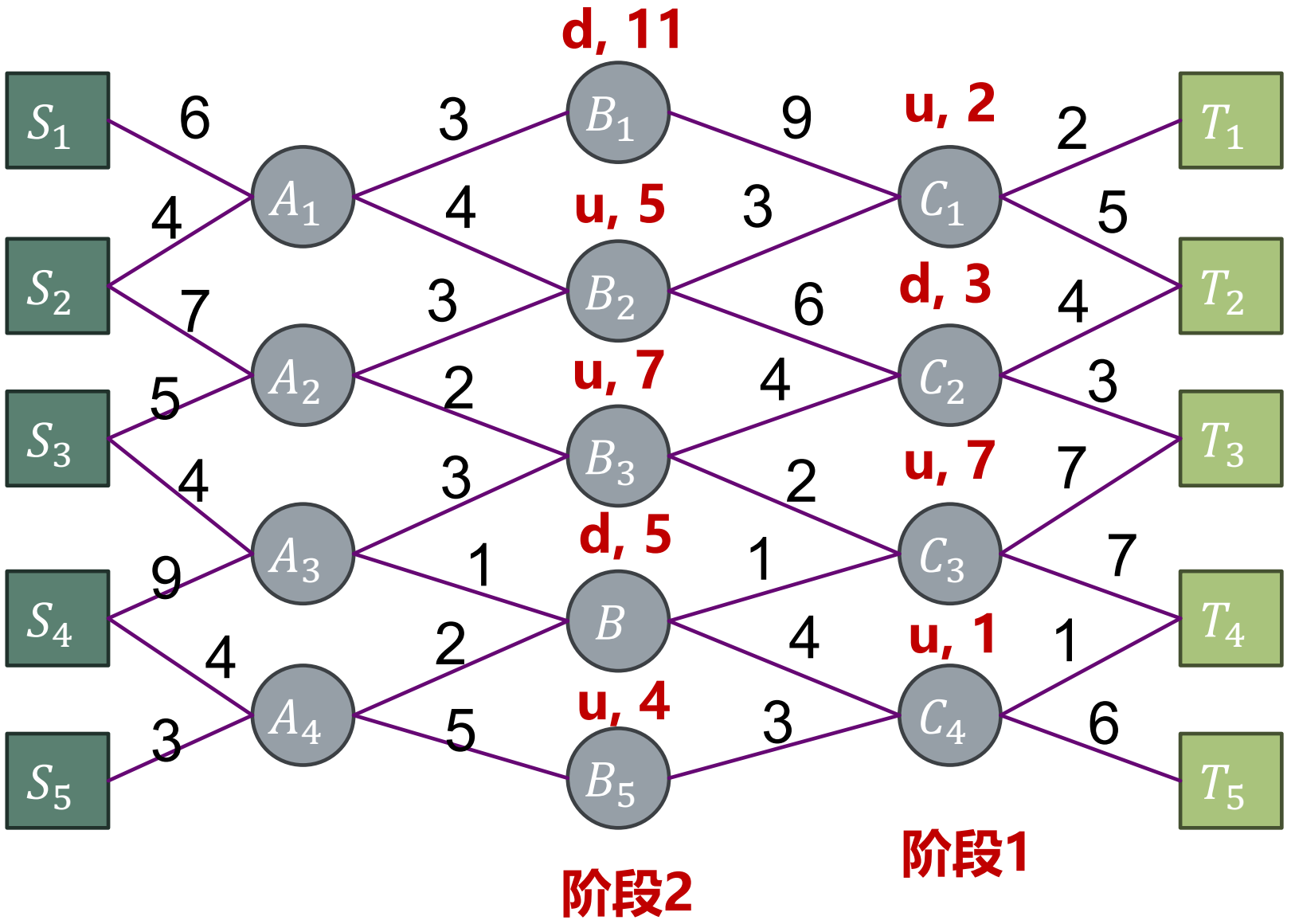
- 穷举算法：计算每一天从某个起点到某个终点的路径的长度，找出其中最小的。
- 在上述例子中，如果网络的层数是k，则路径条数接近2^k
- 动态规划算法：多阶段决策
  - ① 一个大问题可以分解为若干规模更小的子问题；
  - ② 不同子问题之间存在重叠，同一个子问题的结果会被多次使用。

	分治	动态规划
都会做什么	分解成子问题	分解成子问题
子问题关系	通常相互独立	存在重叠
一个子问题结果	通常使用一次	可能被反复使用
核心思想	分而治之	算一次，反复用

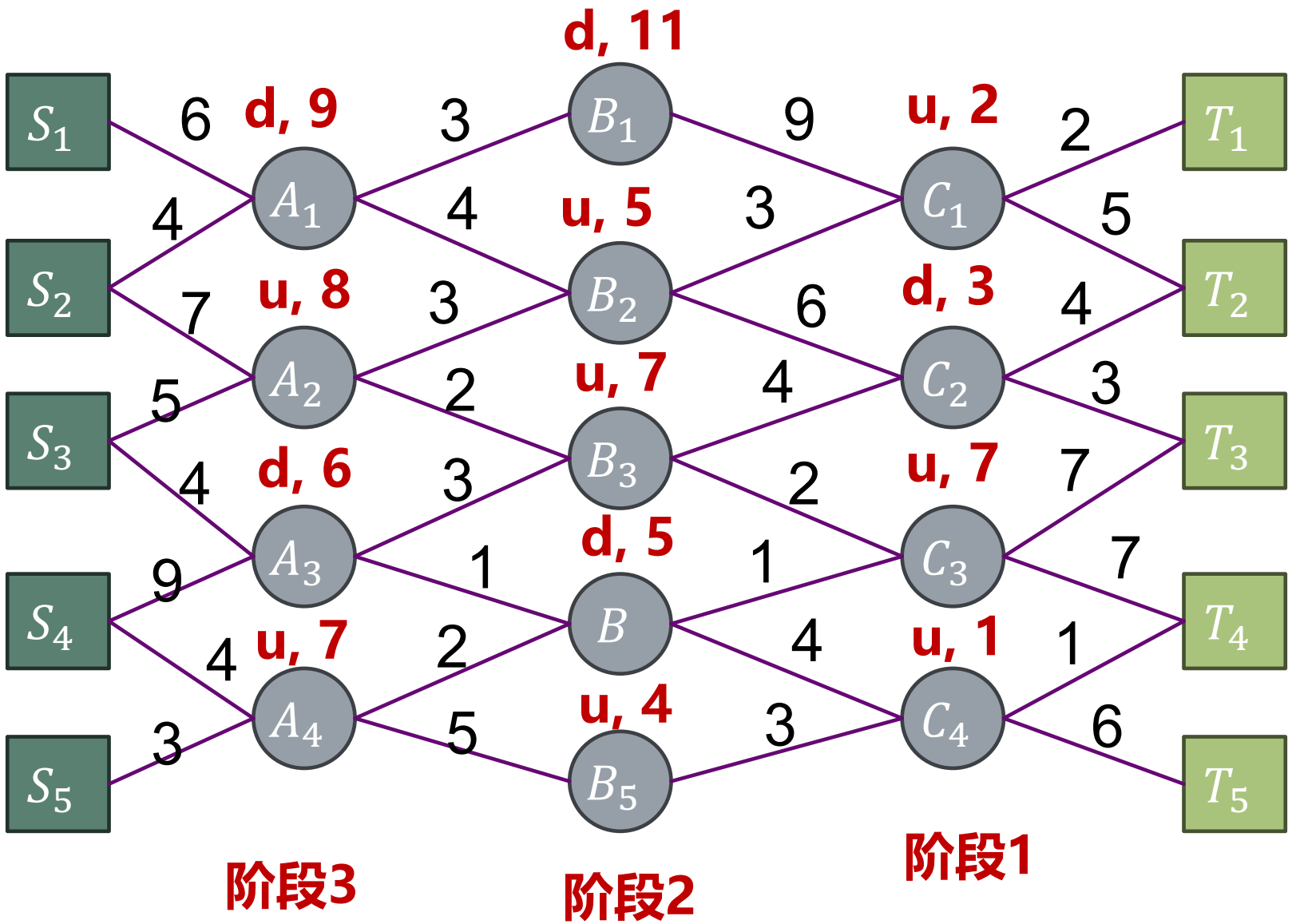
例子


S_1
S_2
S_3
S_4
S_5
A_1
A_2
A_3
A_4
B_1
B_2
B_3
B
B_5
C_1
C_2
C_3
C_4
T_1
T_2
T_3
T_4
T_5
6
4
7
5
4
9
4
3
3
4
3
2
1
2
5
9
4
3
2
4
3
6
4
2
1
4
3
u, 2
d, 3
u, 7
u, 1
2
5
4
3
7
7
1
6
阶段1

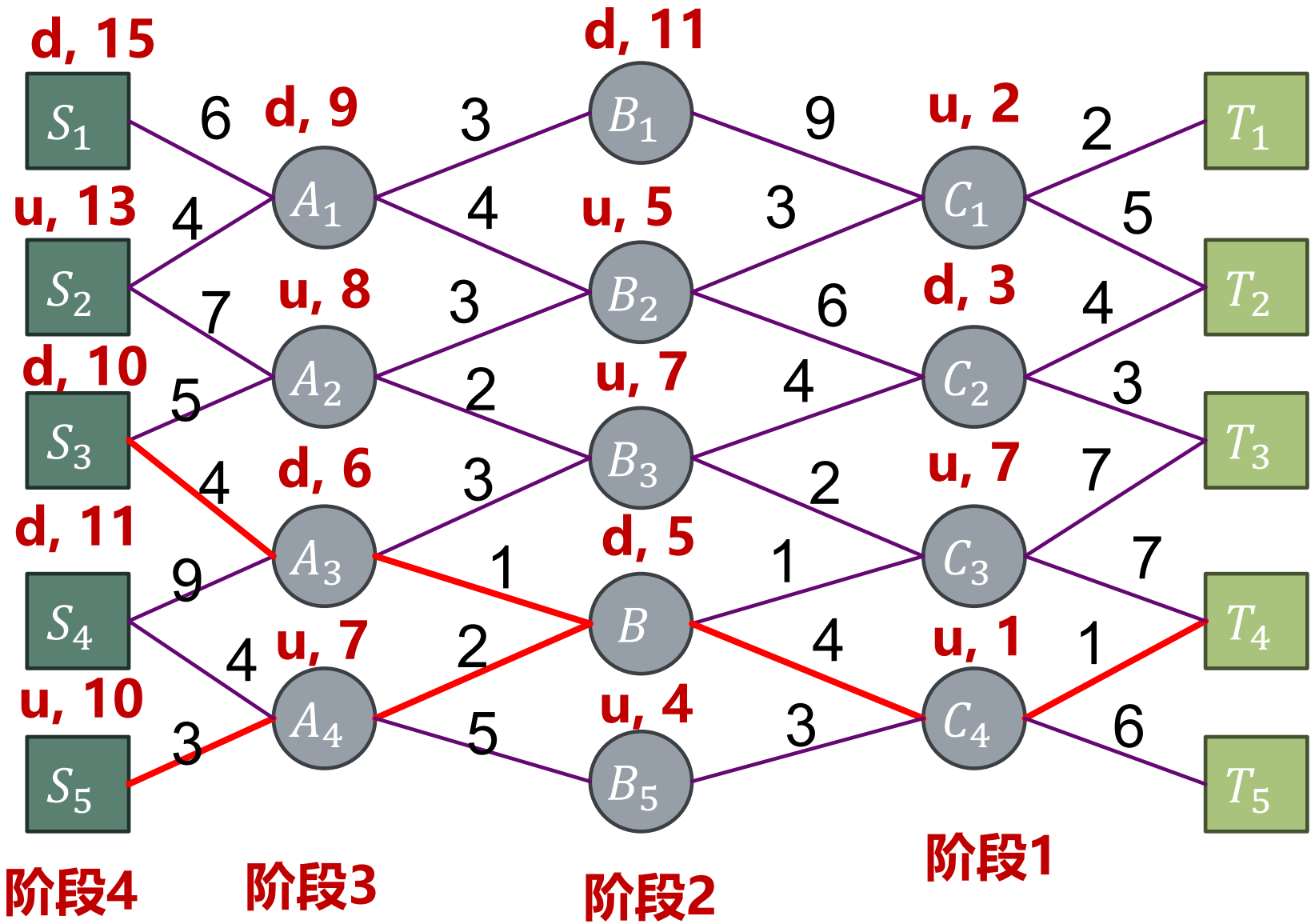
例子


d, 11
u, 2
u, 5
d, 3
u, 7
u, 7
d, 5
u, 1
u, 4
S_1
S_2
S_3
S_4
S_5
A_1
A_2
A_3
A_4
B_1
B_2
B_3
B
B_5
C_1
C_2
C_3
C_4
T_1
T_2
T_3
T_4
T_5
6
4
7
5
4
9
4
3
3
4
3
2
3
1
2
5
9
3
6
4
2
1
4
3
2
5
7
4
3
7
7
1
6
阶段2
阶段1

例子


d, 11
S_1
S_2
S_3
S_4
S_5
d, 9
u, 8
d, 6
u, 7
A_1
A_2
A_3
A_4
u, 5
u, 7
d, 5
u, 4
B_1
B_2
B_3
B
B_5
u, 2
d, 3
u, 7
u, 1
C_1
C_2
C_3
C_4
T_1
T_2
T_3
T_4
T_5
阶段3
阶段2
阶段1

例子


d, 15
d, 11
d, 9
u, 2
u, 13
u, 5
u, 8
d, 3
d, 10
u, 7
d, 6
u, 7
d, 11
d, 5
u, 7
u, 1
u, 10
u, 4
S_1
S_2
S_3
S_4
S_5
A_1
A_2
A_3
A_4
B_1
B_2
B_3
B
B_5
C_1
C_2
C_3
C_4
T_1
T_2
T_3
T_4
T_5
6
3
9
2
4
4
3
5
7
3
6
4
5
2
4
2
3
7
9
1
1
7
4
2
4
1
3
5
3
6
阶段4
阶段3
阶段2
阶段1

子问题界定

- 后边界不变，前边界前移

S_i A_j B_k C_l T_m

子问题界定

- 后边界不变，前边界前移



S_i A_j B_k C_l T_m

子问题界定

- 后边界不变，前边界前移



S_i A_j B_k C_l T_m

子问题界定

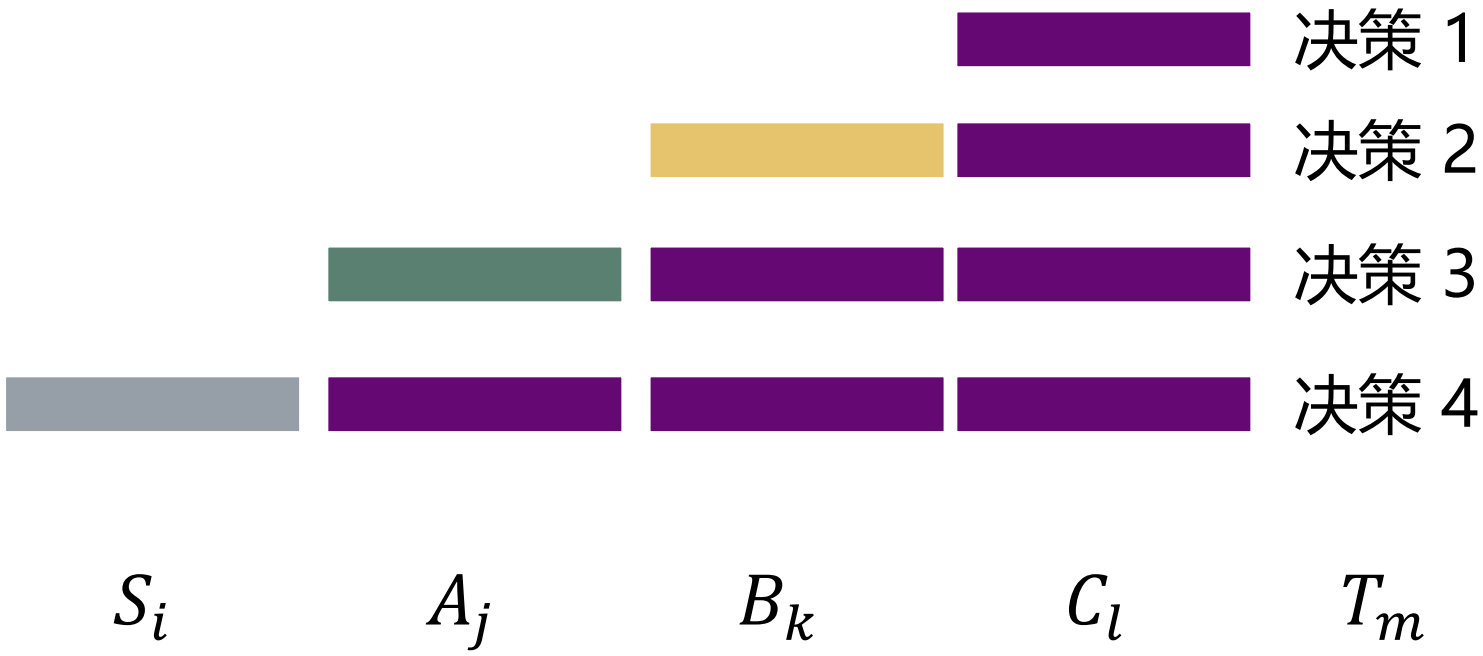
- 后边界不变，前边界前移



S_i A_j B_k C_l T_m

子问题界定

- 后边界不变，前边界前移



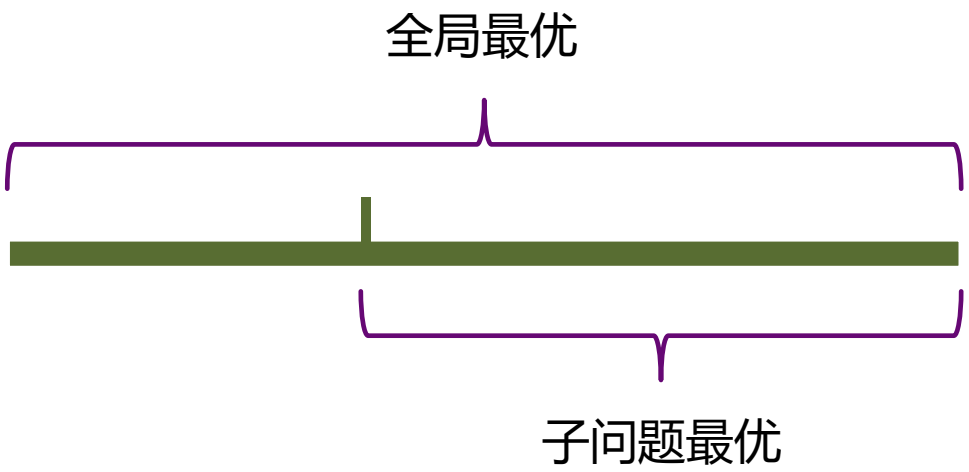
最短路径的依赖关系

- $F(C_l) = \min_m\{C_l T_m\}$ 决策1
- $F(B_k) = \min_l\{B_k C_l + F(C_l)\}$ 决策2
- $F(A_j) = \min_k\{A_j B_k + F(B_k)\}$ 决策3
- $F(S_i) = \min_j\{S_i A_j + F(A_j)\}$ 决策4

- 所谓的“界定”，其实是：找一个边界，使得边界之后的问题能够独立成为一个完整问题
- “好的”子问题必须满足：一旦我知道当前处于哪个状态，前面具体是怎么走到这里的，就可以不再关心

最优子结构

- 如果一个问题的最优解，可以由若干个子问题的最优解拼出来，那么它具有最优子结构。
- 换句话说：最优方案里用到的那一段，也不能有更好的替代方案。
- **例子：**如果一条路是 A到 D的最短路，那么它经过的B到 D那一段，也必须是 B到D的最短路。



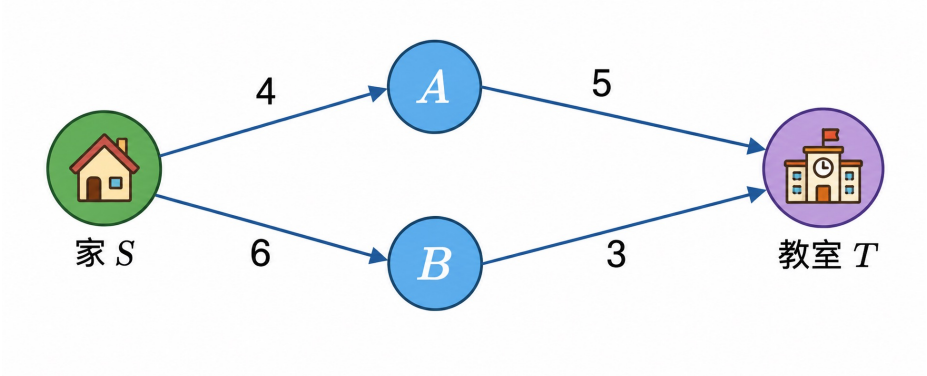
违反最优子结构

- **最优子结构的关键是：** 子问题的最优解，能否安全地替换回原问题？



问题建模：什么叫“最优”？

- 从家到教室可能有很多条路线，不同路线所花的时间也不同。
- 如果我们希望找到“最好”的路线，首先需要明确：这里的“最好”到底指什么？
- 在这个问题中，我们关心的是**总通行时间最短**，因此问题可以表述为：
- **从起点 S到终点 T，寻找一条总时间最短的路线。**



优化目标：最小化总时间

状态定义：我们到底需要解决哪些问题？

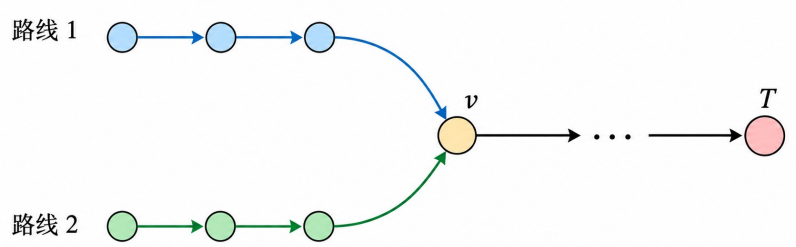
① 从大问题到子问题

- 为了求出从家到教室 T的最短时间，不必直接枚举所有完整路线。
- 可以先求一些更小的问题：从家到 A、B……的最短时间。
- 因此，把“从家到某个位置 v 的最短时间”作为一个子问题。
- 定义状态

$$\boxed{dp[v] = \text{从起点 } S \text{ 到位置 } v \text{ 的最短时间}}$$

- 原问题求：$\boxed{dp[T]}$

② 为什么只记录当前位置 v ？

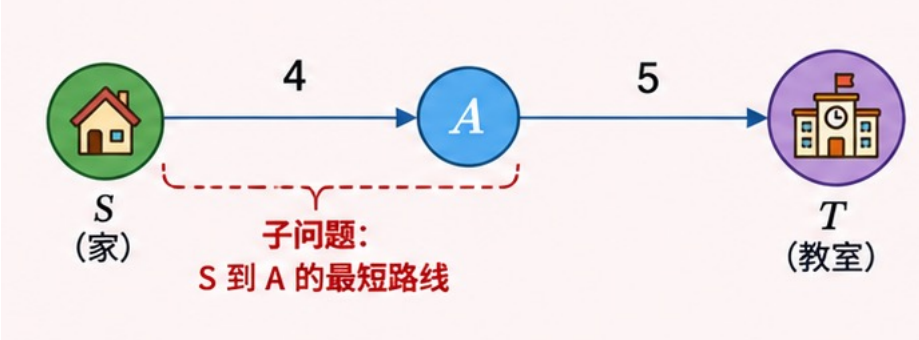


- 两条不同的路线都到达了 v，**接下来从v到 T 怎么走最好，还需要知道前面走的是哪条路线吗？不需要。**
- **无后效性：当前状态确定后，后续决策不再依赖到达该状态之前的具体历史。**

状态已经能够完整描述子问题了。接下来还要确认：每个子问题只保留“最优解”，真的够吗？

最优子结构：子问题的最优解能否找到？是否有用？

- 假设从 S经过 A到达 T 是一条最短路线。那么，从 S 到 A的这一段也必须是到达 A的最短路线。否则，如果存在一条更短路线到达 A，我们就可以把原来的前半段替换掉，使整条 $S \to T$路线变得更短。这与原路线已经最短相矛盾，因此原问题的最优解可以建立在子问题的最优解之上。



原问题的最优解，可以由相应子问题的最优解构成

- 既然到 A、到 B的最优结果都可以被利用，那么怎样把它们组合成到 T的最优结果？

状态转移：怎样由小问题得到大问题？

- 到达教室T的最后一步可能从不同的位置出发。
  - 如果最后一步从 A到 T，总时间就是“到 A的最短时间”再加上 $A \to T$的时间
  - 如果从B到 T，也是同样的道理。
- 因此，只需要比较这些可能的来源，选择总时间最小的一种。
- 建立当前状态与前面状态之间的**状态转移关系**。
- 如果数据设成：$S \to A = 4, A \to T = 5, S \to B = 6, B \to T = 3$
- 那么：$dp[A] = 4, dp[B] = 6$
- 所以：$dp[T] = \min\{dp[A] + 5,\ dp[B] + 3\} = \min\{9,9\} = 9$

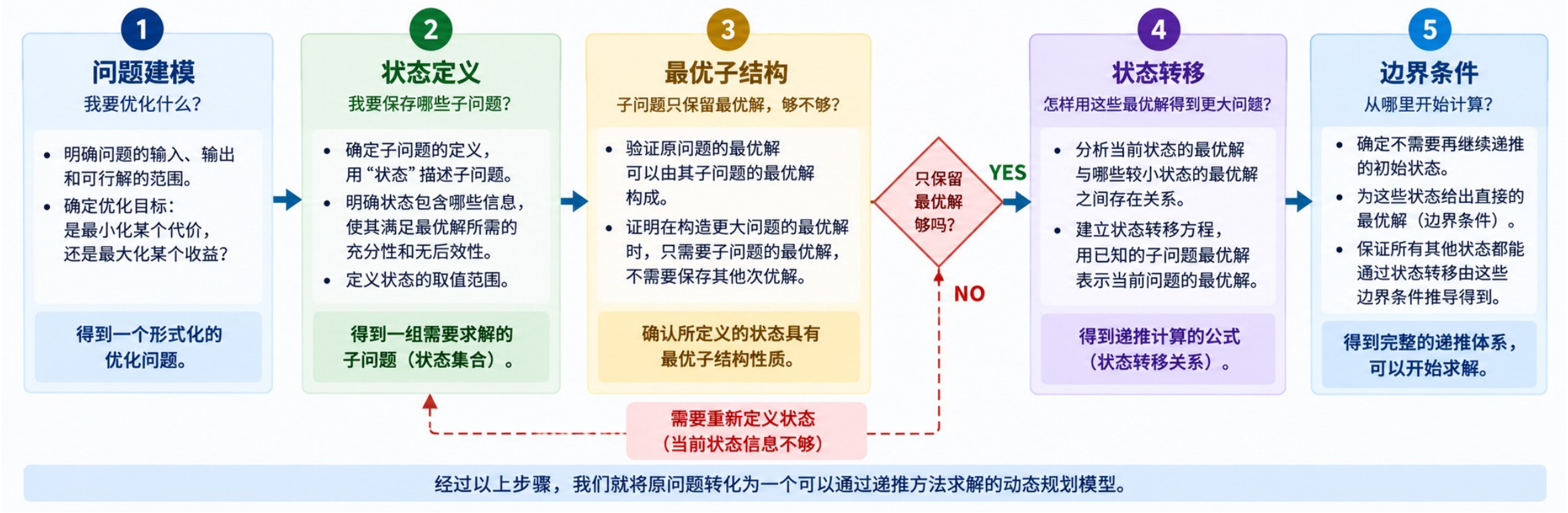
- 这里还有一个问题：$dp[A]$ 又是从哪里算出来的？一直往前推，总得有一个起点。

边界条件：递推从哪里开始?

- 状态转移告诉我们怎样由前面的状态计算后面的状态，但这种递推不能无限向前。
- 对于起点 S，答案不需要再依赖其他状态，因为从家走到家本身不需要任何时间。
- 因此，我们可以直接规定 $dp[S] = 0$，再从这个已知结果出发，逐步计算其他状态。
- **这种能够直接确定、不需要继续递推的最小问题，就是动态规划的边界条件。**

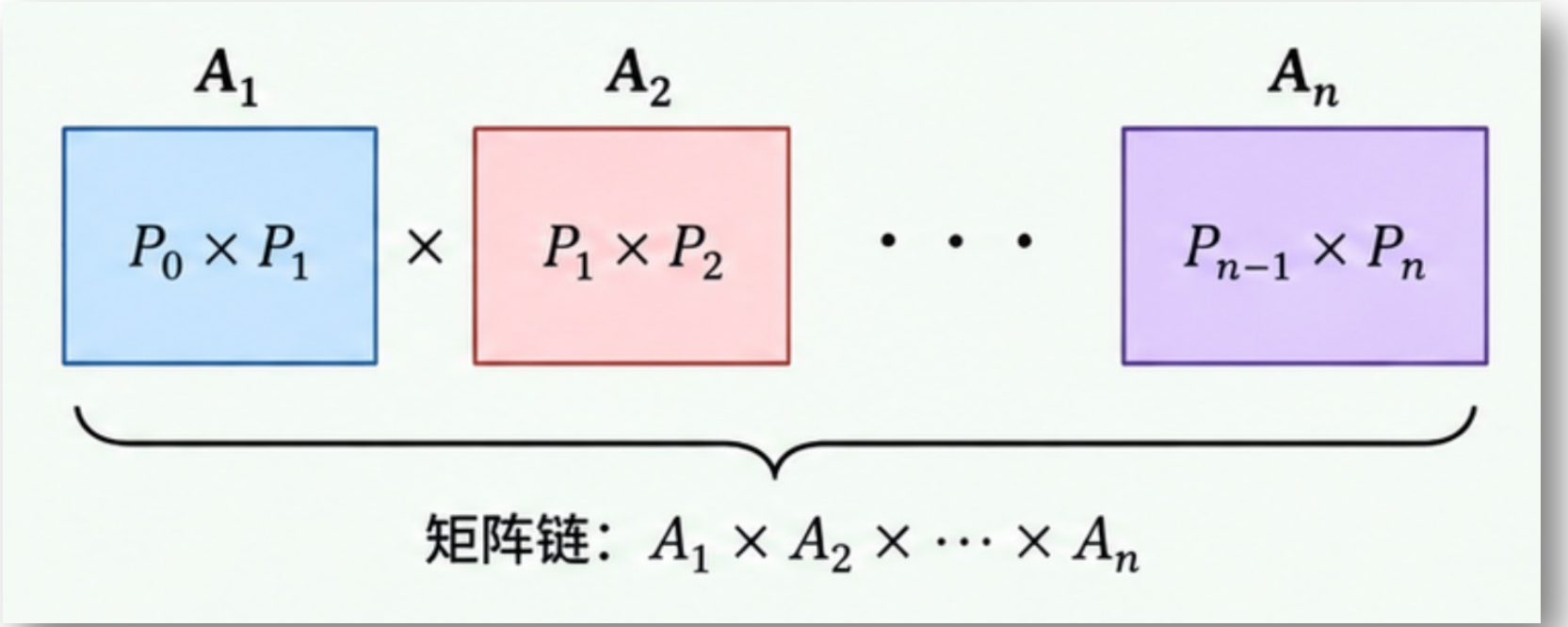
$$\boxed{dp[S] = 0} \rightarrow dp[A] = dp[S] + 4 \rightarrow dp[T] = \min(dp[A] + 5,\ dp[B] + 3)$$

动态规划建模流程


1
问题建模
我要优化什么？
明确问题的输入、输出和可行解的范围。
确定优化目标：是最小化某个代价，还是最大化某个收益？
得到一个形式化的优化问题。
2
状态定义
我要保存哪些子问题？
确定子问题的定义，用“状态”描述子问题。
明确状态包含哪些信息，使其满足最优解所需的充分性和无后效性。
定义状态的取值范围。
得到一组需要求解的子问题（状态集合）。
3
最优子结构
子问题只保留最优解，够不够？
验证原问题的最优解可以由其子问题的最优解构成。
证明在构造更大问题的最优解时，只需要子问题的最优解，不需要保存其他次优解。
确认所定义的状态具有最优子结构性质。
只保留最优解够吗？
YES
NO
4
状态转移
怎样用这些最优解得到更大问题？
分析当前状态的最优解与哪些较小状态的最优解之间存在关系。
建立状态转移方程，用已知的子问题最优解表示当前问题的最优解。
得到递推计算的公式（状态转移关系）。
5
边界条件
从哪里开始计算？
确定不需要再继续递推的初始状态。
为这些状态给出直接的最优解（边界条件）。
保证所有其他状态都能通过状态转移由这些边界条件推导得到。
得到完整的递推体系，可以开始求解。
需要重新定义状态（当前状态信息不够）
经过以上步骤，我们就将原问题转化为一个可以通过递推方法求解的动态规划模型。

问题1：矩阵链相乘问题

- **问题：**设$A_1, A_2, \ldots, A_n$为矩阵序列， A_i为 $P_{i-1} \times P_i$ 阶的矩阵，$i = 1,2, \ldots, n$. 确定矩阵的乘法顺序，使得元素相乘的总次数最少
- **输入：**向量$P =< P_0, P_1, \ldots, P_n >$，其中$P_0, P_1, \ldots, P_n$为$n$个矩阵的行数和列数
- **输出：**矩阵链乘法加括号的位置



矩阵相乘基本运算次数

- 矩阵A: i 行 j 列，B: j 行 k 列，以元素相乘作为基本运算，计算AB的工作量

$$\begin{pmatrix} \cdots & \cdots & \cdots \\ a_{t1} & \cdots & a_{tj} \\ \cdots & \cdots & \cdots \end{pmatrix} \begin{pmatrix} \cdots & b_{1s} & \cdots \\ \cdots & \cdots & \cdots \\ \cdots & b_{js} & \cdots \end{pmatrix} \quad \begin{pmatrix} \cdots & \cdots & \cdots \\ \cdots & c_{ts} & \cdots \\ \cdots & \cdots & \cdots \end{pmatrix}$$

$$c_{ts} = a_{t1}b_{1s} + a_{t2}b_{2s} + \cdots + a_{tj}b_{js}$$

- 矩阵AB: i 行 k 列，计算每个元素需要做 j 次乘法，总计乘法次数为 ***ijk***

例子

- 当$P =< 10, 100, 5, 50 >$

$$A_1: 10\times100, A_2: 100\times5, A_3: 5\times50$$

- 乘法次序
- $(A_1A_2)A_3 = 10\times100\times5 + 10\times5\times50 = 7500$
- $A_1(A_2\ A_3) = 10\times100\times50 + 100\times5\times50 = 75000$

- **第一种次序计算次数最少**

用动态规划求解

- 问题建模：优化目标非常明确，最小化标量乘法总次数
- **在所有合法的加括号方式中，寻找计算代价最小的一种**

- **如何定义状态？**

状态定义

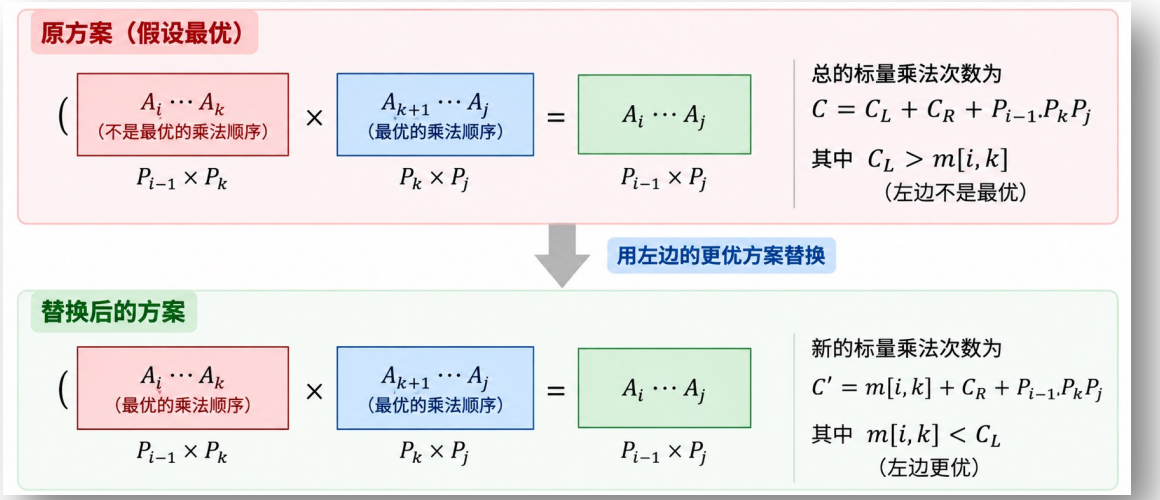
- 要计算整条矩阵链：$A_1A_2\cdots A_n$
- 可以先考虑更短的一段，例如：$A_iA_{i+1}\cdots A_j$.
- 所以最自然的子问题就是：**矩阵 A_i到 A_j这一段怎样相乘最省？**
- 状态定义：

$$\boxed{m[i,j]=A_iA_{i+1}\cdots A_j \text{ 的最少标量乘法次数}}$$

- 无后效性
  - $A_i\cdots A_j$，不管内部怎样加括号，最终得到的矩阵维度始终都是$P_{i-1}\times P_j$
  - 也就是说，**一旦知道区间 $[i,j]$，这个子问题后面需要的信息就已经确定了**。

最优子结构

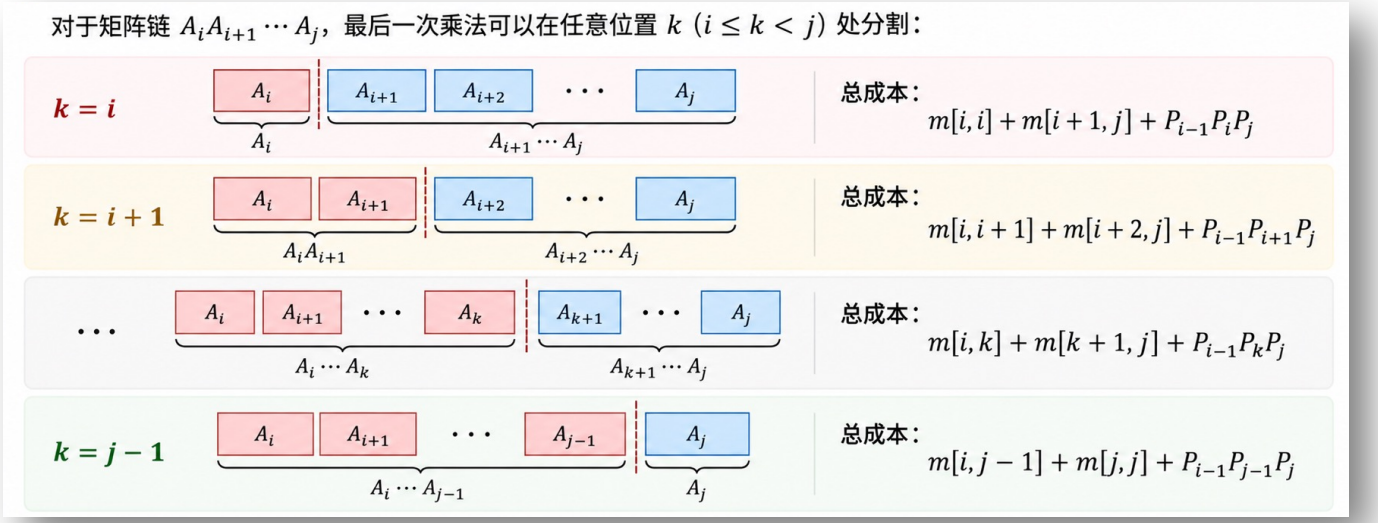
- 假设$A_i \cdots A_j$的最优括号方式，最后一次乘法在k处分开：$(A_i \cdots A_k)(A_{k+1} \cdots A_j)$
- 问：如果整个$A_i \cdots A_j$已经是最优方案，左边 $A_i \cdots A_k$可以不是自己的最优方案吗？



- 因此：$A_i \cdots A_j$ 的最优解一定由$A_i \cdots A_k$ 的最优解和$A_{k+1} \cdots A_j$ 的最优解组成。

状态转移

- 区间：$A_iA_{i+1}\cdots A_j$，最后一次乘法一定要在某个 k 处分开：$(A_i\cdots A_k)(A_{k+1}\cdots A_j)$其中：$i\le k<j$
  - 左边最少需要：$m[i,k]$
  - 右边最少需要：$m[k+1,j]$



- 状态转移方程：$\boxed{m[i,j]=\min_{i\le k<j}\{m[i,k]+m[k+1,j]+P_{i-1}P_kP_j\}}$
- 为了最后输出括号位置，还要额外记录：$\boxed{s[i,j]=\arg\min_{i\le k<j} m[i,k]+m[k+1,j]+P_{i-1}P_kP_j}$

边界条件

- 最小的矩阵链只有一个矩阵：A_i，一个矩阵本身根本不需要做矩阵乘法。所以：$\boxed{m[i,i]=0}$

```
MatrixChain(i, j):
    if i == j:                  // 边界条件
        return 0
    best = ∞
    for k = i ... j-1:          // 枚举分割位置 k
        cost =
            MatrixChain(i, k)    // 左子问题
          + MatrixChain(k+1, j)  // 右子问题
          + P[i-1] * P[k] * P[j]// 最后一次矩阵乘法

        best = min(best, cost)   // 选择最优分割

    return best
```

算法复杂度分析

- $T(n) \geq \begin{cases} O(1) & n = 1 \\ \sum_{k=1}^{n-1}(T(k) + T(n-k) + O(1)) & n > 1 \end{cases}$
- 所以：$\boxed{T(n) = 2\sum_{k=1}^{n-1} T(k) + \Theta(n)}$，忽略低阶项，可以看成：$T(n) \approx 2\sum_{k=1}^{n-1} T(k)$
- 而：$T(n-1) \approx 2\sum_{k=1}^{n-2} T(k)$，因此：

$$T(n) \approx 2\left(\sum_{k=1}^{n-2} T(k) + T(n-1)\right)$$
$$\approx T(n-1) + 2T(n-1)$$
$$= 3T(n-1)$$

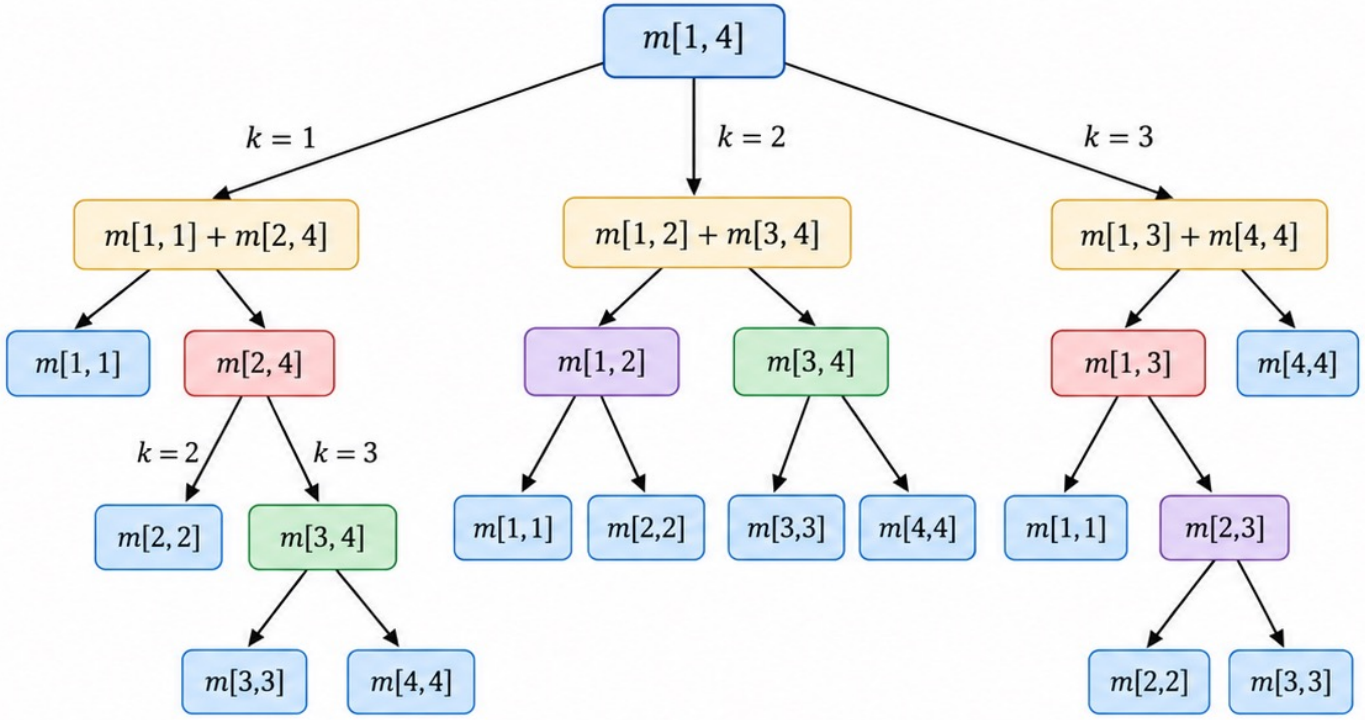
- 所以：$T(n) \approx 3T(n-1) \approx 3^2 T(n-2) \approx \cdots \approx 3^{n-1} T(1)$

$$\boxed{T(n) = \Theta(3^n)}$$

为什么这么慢？

子问题重复计算问题

求 $m[1,4]$ 时，需要尝试所有分割位置 $k=1,2,3$，因此会递归地求解更小的子问题。



可以看到，很多子问题被重复计算：

- $m[3,4]$ 出现在左子树和中间子树中；
- $m[1,2]$ 在中间子树和右子树中会出现；
- $m[2,3]$ 在右子树中出现，随着规模增大，会在更多分支中出现；
- 更大的 n 时，重复计算的情况会非常严重。

出现多次的子问题（示例）

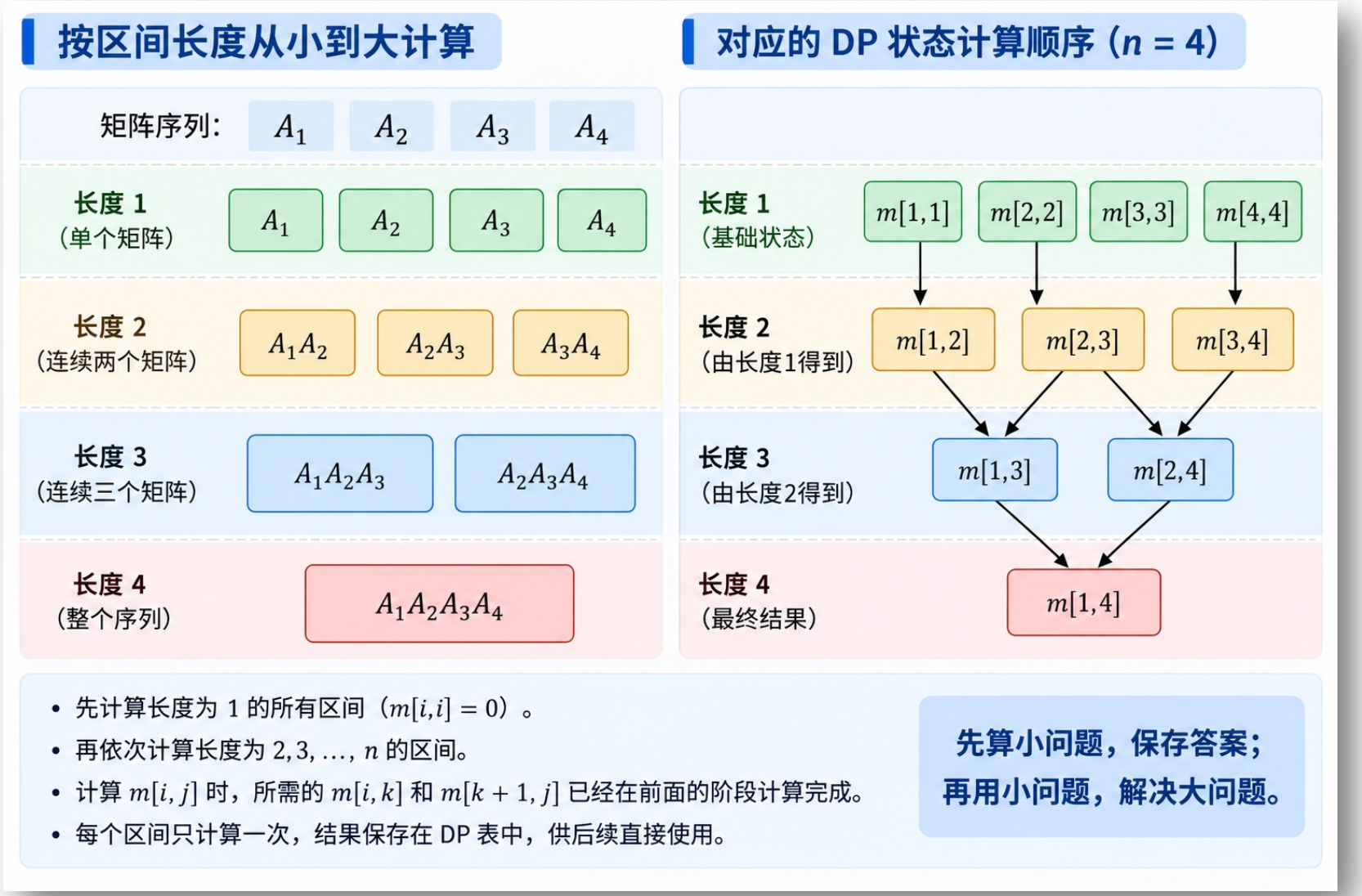
子问题	出现的位置
$m[3,4]$	左子树、中间子树
$m[1,2]$	中间子树、右子树
$m[2,3]$	右子树（在更大规模时会多次出现）
…	…

同一个子问题会在不同的递归分支中被重复计算，导致大量的重复工作，这是朴素递归效率低的本质原因。

动态规划：自底向上迭代求解

- **核心思想：子问题只计算一次，并把结果保存下来，供更大的子问题直接使用。**
- **求解过程：**
  - **从最小的子问题开始**
    - 例如矩阵链中：$m[i,i]=0$
  - **按照子问题规模逐步扩大**
    - 先求长度为 2 的矩阵链，再求长度为 3、4，直到整条矩阵链。$1 \to 2 \to 3 \to \cdots \to n$
  - **保证依赖的子问题已经求**
    - 在计算$m[i,j]$时，需要的$m[i,k], m[k+1,j]$都已经在前面的阶段计算完成。
  - **将结果保存到 DP 表（状态表）**
    - 一个状态只计算一次，后面需要时直接查表。
- 先算小问题，保存答案；再用小问题，解决大问题
- 像盖楼一样：先打地基，再一层一层向上建。每一层都建立在已经完成的下面几层之上。

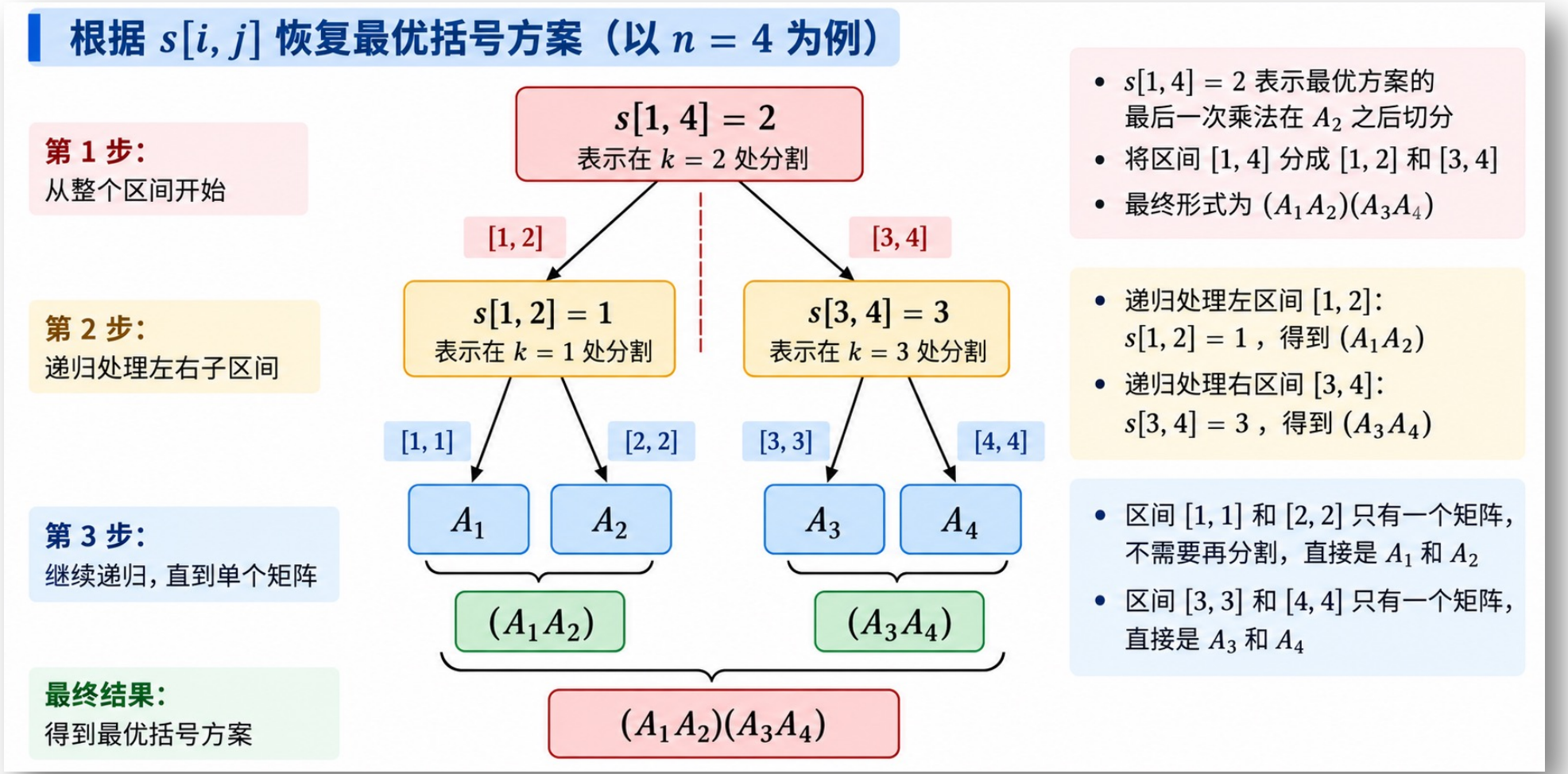
动态规划：自底向上迭代求解



解的追踪：最优方案是什么?

- DP 表 $m[i,j]$告诉我们**最少需要多少次乘法**，但仅有最优值，还不知道括号应该放在哪里。在计算 $m[i,j]$时，我们不仅记录最小代价，还记录产生该最小代价的切分位置 k。
- 定义：$\boxed{s[i,j] = \text{区间 } [i,j] \text{ 的最优切分位置}}$
- 计算时：$m[i,j] = \min_k \{m[i,k] + m[k+1,j] + P_{i-1}P_kP_j\}$
- 同时：$\boxed{s[i,j] = \arg\min_k \{m[i,k] + m[k+1,j] + P_{i-1}P_kP_j\}}$
- 最后：从 $s[1,n]$ 出发，不断分成左右两个区间，直到只剩单个矩阵，就能恢复完整的最优括号方案。

解的追踪


根据 $s[i, j]$ 恢复最优括号方案（以 $n = 4$ 为例）
第 1 步：
从整个区间开始
第 2 步：
递归处理左右子区间
第 3 步：
继续递归，直到单个矩阵
最终结果：
得到最优括号方案
$s[1, 4] = 2$
表示在 $k = 2$ 处分割
[1, 2]
[3, 4]
$s[1, 2] = 1$
表示在 $k = 1$ 处分割
$s[3, 4] = 3$
表示在 $k = 3$ 处分割
[1, 1]
[2, 2]
[3, 3]
[4, 4]
A_1
A_2
A_3
A_4
(A_1A_2)
(A_3A_4)
$(A_1A_2)(A_3A_4)$
• $s[1, 4] = 2$ 表示最优方案的最后一次乘法在 A_2 之后切分
• 将区间 [1, 4] 分成 [1, 2] 和 [3, 4]
• 最终形式为 $(A_1A_2)(A_3A_4)$
• 递归处理左区间 [1, 2]：$s[1, 2] = 1$，得到 (A_1A_2)
• 递归处理右区间 [3, 4]：$s[3, 4] = 3$，得到 (A_3A_4)
• 区间 [1, 1] 和 [2, 2] 只有一个矩阵，不需要再分割，直接是 A_1 和 A_2
• 区间 [3, 3] 和 [4, 4] 只有一个矩阵，直接是 A_3 和 A_4

算法伪代码

```
MatrixChainOrder(P, n):
    # ① 初始化：单个矩阵不需要乘法
    for i = 1 to n:
        m[i][i] = 0
    # ② 按矩阵链长度从小到大计算
    for L = 2 to n:
        # ③ 枚举区间 [i, j]
        for i = 1 to n - L + 1:
            j = i + L - 1

            m[i][j] = ∞
            # ④ 枚举最后一次乘法的切分位置 k
            for k = i to j - 1:

                cost = (
                    m[i][k]                          # 左子问题
                    + m[k + 1][j]                    # 右子问题
                    + P[i - 1] * P[k] * P[j]         # 最后一次乘法
                )

                # ⑤ 保存当前最优值和最优切分点
                if cost < m[i][j]:
                    m[i][j] = cost
                    s[i][j] = k

    return m[1][n], s
```

```
PrintOptimalParens(s, i, j):

    # 只有一个矩阵，不再切分
    if i == j:
        print A[i]
        return

    # 查找最优切分点
    k = s[i][j]

    print "("

    # 左半部分
    PrintOptimalParens(s, i, k)

    # 右半部分
    PrintOptimalParens(s, k + 1, j)

    print ")"
```

时间复杂度分析

- **1. 有多少个子问题？**
  - 状态：$m[i,j],\quad 1 \le i \le j \le n$
  - 不同区间数量约为：$\boxed{O(n^2)}$
- **2. 每个子问题要做多少工作？**
  - 计算 $m[i,j]$时，需要枚举切分位置：$k = i, i+1, \dots, j-1$
  - 最多：$\boxed{O(n)}$
- **3. 总时间复杂度**
  - $\boxed{O(n^2) \times O(n) = O(n^3)}$
  - 更精确地：$T(n) = \sum_{L=2}^{n} (n - L + 1)\,(L - 1) = \Theta(n^3)$
- 查找解的复杂度：
  - 若最优切分位置把问题分成 k 个和 $n-k$个矩阵：$T(n) = T(k) + T(n-k) + \Theta(1) = \boxed{O(n)}$

区间长度	区间个数
1	n
2	$n-1$
3	$n-2$
$\vdots$	$\vdots$
n	1

- 因此状态总数为：
  $n + (n-1) + (n-2) + \cdots + 1$

例子

输入 $P = \langle 10, 30, 5, 60, 10 \rangle$，$n = 4$

矩阵链：$A_1 A_2 A_3 A_4$，其中

A_1: 10×30，A_2: 30×5，A_3: 5×60，A_4: 60×10

$m[i, j]$：计算 $A_i \cdots A_j$ 的最少标量乘法次数

$s[i, j]$：最优切分点 k（在 A_k 和 A_{k+1} 之间切分）

最少乘法次数表 $m[i, j]$

i \ j	1	2	3	4
1	0	1500	4500	2500
2	–	0	9000	4500
3	–	–	0	3000
4	–	–	–	0

最优切分点表 $s[i, j]$

i \ j	1	2	3	4
1	–	1	1	3
2	–	–	2	2
3	–	–	–	3
4	–	–	–	–

最优解：最少乘法次数 $m[1,4] = 2500$，对应的最优括号结构为 $(A_1(A_2A_3))A_4$

问题2：连写分数

- 给定一个长度为 n 的十进制数字串$S = s_1 s_2 \cdots s_n$，现有两类题目：
  - m道**特殊题**，第i道特殊题分值的上限为 v_i，每道特殊题必须且只能使用一次；
  - **普通题分值**的上限均为t，普通题可以使用任意多次。
- 需要将数字串 S按原顺序划分为若干个**非空连续子串**：$S = x_1 x_2 \cdots x_k$
  - 每个子串 x_j表示一个十进制非负整数 a_j，要求：
  - 单独的 "0" 是合法数字；
  - 若 $|x_j| > 1$，则 x_j的首位不能为 0；
  - 每个 a_j必须分配给一道题目；
  - 如果分配给特殊题 i，必须满足$a_j \le v_i$;
  - 如果分配给普通题，必须满足$a_j \le t$;
  - 每道特殊题必须**恰好分配一个数字**；普通题可以分配任意多个数字。
- 目标是最大化所有划分所得数字之和：$\boxed{\max \sum_{j=1}^{k} a_j}$
- 如果不存在满足要求的划分，可以规定输出 -1。
- 输入规模：$\boxed{1 \le n \le 10^5,\ m \le 7,\ 0 \le v_i,\ t \le 9999}$

例子

- **数字串：**12345，**特殊题目2道（**$m = 2$**），分值：，**$v_1 = 30$，$v_2 = 12$，**普通题分值：**t = 9
- 一种划分：12 | 3 | 45
  - 12 → 分给特殊题 $v_1 = 30$✓
  - 3 → 分给特殊题 $v_2 = 12$✓
  - 45 → 分给普通题 $t = 9$✗ $(45 > 9$，不合规)
- 另一种划分
  - 1 → 分给普通题 $t = 9$✓
  - 23 → 分给特殊题 $v_1 = 30$✓
  - 4 → 分给普通题$t = 9$✓
  - 5 → 分给特殊题 $v_2 = 12$✓
  - 总和 $= 1 + 23 + 4 + 5 = 33$

问题建模

- **目标**：将数字串划分为若干连续段，最大化所有段对应数字之和

$$\max \sum_{j} x_j$$

- **约束**：
  - **合法数字**：单独的 0 合法；长度大于 1 时不能有前导零。
  - **完整划分**：所有段均为非空连续子串，按原顺序覆盖整个字符串，**不重不漏**。
  - **题目分配**：每一段必须且只能分配给一道题目。
  - **容量约束**：若数字 x分配给上限为 c的题目，则$x \leq c$
  - **特殊题**：m道特殊题，每道**恰好使用一次**。
  - **普通题**：上限为 t，可以使用任意多次。

状态定义

- 处理数字串时，后面的决策取决于两件事：
  - **已经处理到字符串的哪个位置**
  - **哪些特殊题已经使用过**
- 因此状态定义：$\boxed{dp[i][mask] = \text{前 } i \text{ 个字符已经完成合法划分时，能够得到的最大数字和}}$
  - $i = 0,1,\dots,n$，表示已经处理完字符串前i个字符。mask 用二进制记录特殊题的使用情况：$mask \in [0, 2^m)$
  - 第 j位：$\begin{cases} 1, & \text{特殊题 } j \text{ 已使用} \\ 0, & \text{特殊题 } j \text{ 尚未使用} \end{cases}$，例如 $m = 4$：$mask = 0101_2$
- **无后效性**
  - 一旦$(i, mask)$确定：剩下哪个字符串还没有处理，确定了；哪些特殊题还可以使用，确定了；普通题始终可以继续使用。
  - 所以：**未来如何划分，不再需要知道前面具体是怎么划分出来的。**

最优子结构

- 假设有两种方案都到达：$(i, mask)$，但前缀收益分别为：$V_1 < V_2$

方案 1 —— 前 i 个字符 $\rightarrow (i, mask) \rightarrow$ 后续方案
收益 V_1
方案 2 —— 前 i 个字符 $\rightarrow (i, mask) \rightarrow$ 后续方案
收益 V_2

- 因为两个方案到达的是**完全相同的状态**：
  - 剩余字符串相同；
  - 剩余特殊题相同；
  - 普通题仍然可无限使用。
- 所以从这里往后的所有可选方案完全一样。于是如果方案 1 能接某个后续方案，方案 2 也一定能接同一个后续方案。
- 假设后续能够再获得 R，那么：$V_1 + R < V_2 + R$
- 所以：**到达同一个状态时，较差的前缀永远不可能成为最终最优解的一部分**。因此我们只需要保留：

$$\boxed{dp[i][mask] = \text{到达状态 } (i, mask) \text{ 的最大收益}}$$

状态转移

- 从：$\boxed{dp[i][mask]}$
- 枚举下一段：$x = S[i+1 \ldots j]$
- 有两种选择：
- $$\begin{aligned} x \le t: &\quad dp[j][mask] \leftarrow \max(dp[j][mask], dp[i][mask] + x) \\ x \le v_q, q \notin mask: &\quad dp[j][mask \cup \{q\}] \leftarrow \max(dp[j][mask \cup \{q\}], dp[i][mask] + x) \end{aligned}$$
- 每次状态转移只做两件事：
  - 确定下一段在哪里结束
  - 再决定把它分给普通题还是某一道尚未使用的特殊题。

状态转移

- $x \le t$:　$dp[j][mask] \leftarrow \max(dp[j][mask], dp[i][mask] + x)$
- $$dp[j][mask] \leftarrow \max\left(\underbrace{dp[j][mask]}_{\text{之前到达这个状态的最好结果}}, \underbrace{dp[i][mask] + x}_{\text{当前这条转移得到的新结果}}\right)$$
- 这里的 max 本质上是在做：**多条路径都能到同一个状态，只保留收益最大的那一条。**
  - 例如原来已经有：$dp[j][mask] = 20$
  - 现在新方案得到：$dp[i][mask] + x = 25$
  - 当然应该把它更新成：$dp[j][mask] = 25$
  - 反过来，如果新方案只有：$dp[i][mask] + x = 18$
  - 那就没有必要把原来的 20 换掉。

边界条件

- **初始化：** $\boxed{dp[0][0] = 0}$
- **其他状态：** $\boxed{dp[i][mask] = -\infty}$
- **目标状态：** $\boxed{dp[n][2^m - 1]}$
- 状态变化：
  - 从“什么都没处理”开始，直到“字符串全部处理完且所有特殊题恰好使用一次”

伪代码

```
# dp[i][mask]：处理完前 i 个字符，已使用特殊题集合为
mask 时的最大收益

dp[*][*] = -∞
dp[0][0] = 0

for i = 0 ... n-1:
    for mask = 0 ... 2^m-1:

        if dp[i][mask] == -∞:
            continue

        x = 0

        # 枚举下一段 S[i...j]，最多 4 位
        for j = i ... min(i+3, n-1):

            # 0 合法，但 01、001 不合法
            if j > i and S[i] == '0':
                break

            x = 10*x + digit(S[j])

            # ① 分给普通题
            if x <= t:
                dp[j+1][mask] =
                    max(dp[j+1][mask],
                        dp[i][mask] + x)

            # ② 分给某个未使用的特殊题
            for q = 0 ... m-1:

                if q not in mask and x <= v[q]:

                    newMask = mask ∪ {q}

                    dp[j+1][newMask] =
                        max(dp[j+1][newMask],
                            dp[i][mask] + x)

answer = dp[n][2^m - 1]
```

算法复杂度

- 状态数：$dp[i][mask]$，其中$0 \le i \le n, 0 \le mask < 2^m$，因此共有：$O(n2^m)$个状态。
- 对每个状态，需要做两件事：
- **枚举下一段数字**
  - 因为$v_q, t \le 9999$所以合法数字最多 4 位，最多枚举：$\boxed{4}$种长度。
- **枚举分配对象**
  - 每个数字可以尝试：1 种普通题；m道特殊题。
  - 因此最多：$\boxed{m+1}$次检查。
- 所以总时间：$T(n,m) = O\big(n \cdot 2^m \cdot 4 \cdot (m+1)\big)$，忽略常数：$\boxed{T(n,m) = O(n2^m m)}$
- **解的恢复**
  - 从$(n, 2^m - 1)$ 不断回溯，每次至少减少一个字符，所以最多回溯 n次：$\boxed{O(n)}$

DP表（例子）

1. 问题描述

数字串： $S =$ “314159” $(n = 6)$

普通题上限： $t = 50$

特殊题上限： $v_0 = 20$，$v_1 = 45$ $(m = 2)$

要求：

- 将 S 划分为若干连续子串，每个子串分配给普通题或一道特殊题。
- **两道特殊题必须各使用一次（且只能使用一次）。**
- 使得总收益最大，并给出具体划分方案。

2. 状态定义

dp[i][mask]：

处理完前 i 个字符，已使用的特殊题集合为 mask 时的最大收益。

mask 用二进制表示（m = 2）：

0（00）：两道特殊题都未使用
1（01）：已使用特殊题 0
2（10）：已使用特殊题 1
3（11）：两道特殊题都已使用

3. 最终答案

最大收益： 86

最优划分与分配：

- “31” → 普通题 （≤ 50）
- “41” → 普通题 （≤ 50）
- “5” → 特殊题 0 （≤ 20）
- “9” → 特殊题 1 （≤ 45）

总收益 = 31 + 41 + 5 + 9 = 86

4. DP 表（最大收益 dp[i][mask]）

i（前 i 个字符）	前缀 $S[1..i]$	mask 0 (00)	mask 1 (01)	mask 2 (10)	mask 3 (11)
0	（空串）	0	−∞	−∞	−∞
1	3	3	3	3	−∞
2	31	31	4	31	4
3	314	35	35	35	35
4	3141	72	45	72	45
5	31415	77	77	77	77
6	314159	86	86	86	**86**

5. 追踪表（pre[i][mask]）

i（前 i 个字符）	前缀 $S[1..i]$	mask 0 (00)	mask 1 (01)	mask 2 (10)	mask 3 (11)
0	（空串）	-	-	-	-
1	3	(0,0,P)	(0,0,S0)	(0,0,S1)	-
2	31	(0,0,P)	(1,0,S0)	(0,0,S1)	(1,1,S1)
3	314	(2,0,P)	(2,0,S0)	(2,0,S1)	(2,2,S0)
4	3141	(2,0,P)	(2,1,P)	(2,0,S1)	(2,1,S1)
5	31415	(4,0,P)	(4,0,S0)	(4,0,S1)	(4,2,S0)
6	314159	(5,0,P)	(5,0,S0)	(5,0,S1)	**(5,1,S1)**

记号说明： (a, b, P)：从状态 (a, b) 转移而来，当前子串分配给普通题
$(a, b, S0)$：从状态 (a, b) 转移而来，当前子串分配给特殊题 0
$(a, b, S1)$：从状态 (a, b) 转移而来，当前子串分配给特殊题 1

6. 最优解的回溯过程 （从 (6, 3) 开始）

步骤	当前状态 (i, mask)	前驱状态 (pre[i][mask])	取出的子串	分配给	该段收益	累计收益
1	(6, 3)	(5, 1, S1)	“9”	特殊题 1	9	86
2	(5, 1)	(4, 0, S0)	“5”	特殊题 0	5	77
3	(4, 0)	(2, 0, P)	“41”	普通题	41	72
4	(2, 0)	(0, 0, P)	“31”	普通题	31	31

从 (6, 3) 逆序回溯得到子串顺序： 9 | 5 | 41 | 31
按原顺序（从左到右）： 31 | 41 | 5 | 9

7. 最终划分方案



普通题 (≤ 50)	普通题 (≤ 50)	特殊题 0 (≤ 20)	特殊题 1 (≤ 45)
收益 31	收益 41	收益 5	收益 9

总收益 = 31 + 41 + 5 + 9 = 86

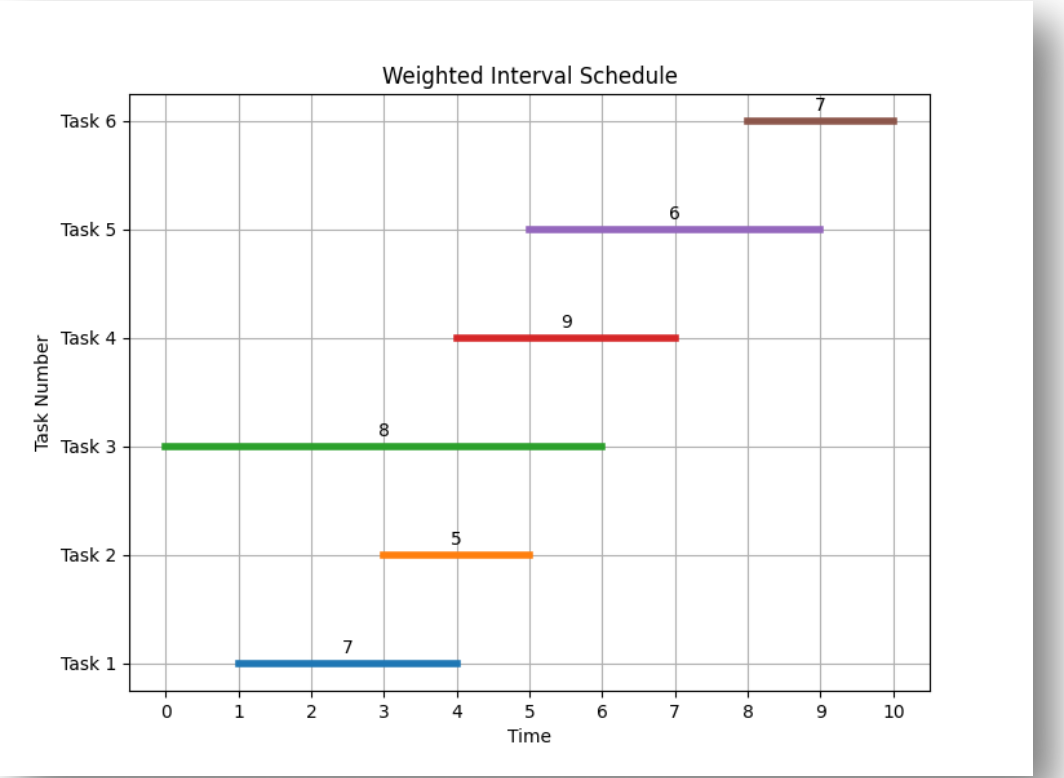
问题3：带权重的区间调度

- 问题：n个任务，任务$j \in [1,n]$开始于时间s_j，结束于时间e_j，权重为w_i，找到一个相互兼容（时间上不重叠）任务的最大权重子集。

- 建模：问题的解是向量$x =< x_1, x_2, \dots x_n >$，表示任务是否在子集中
- 目标函数：$\max \sum_{i=1}^{n} w_i x_i$
- 约束条件1：$x_i + x_j \leq 1, \forall (i,j) \in \{(i,j) | (e_i > s_j) \vee (s_i < e_j)\}$
- 约束条件2：$x_i \in \{0,1\}, \forall i \in [1,n]$

例子

任务编号	开始时间	结束时间	权重
1	1	4	7
2	3	5	5
3	0	6	8
4	4	7	9
5	5	9	6
6	8	10	7



动态规划求解

- 设目标为$OPT(n)$，为从n个任务中能够获取到的最大权重
- 设$p(j)$为在任务j之前的所有任务里面离 j 最近并且兼容的活动（如果没有兼容的活动，则$p(j)=0$）
- 状态：是否要进行任务j
  - 进行任务j获得的最大权重：$OPT(p(j))+w_{\mathrm{j}}$
  - 不进行任务j获得的最大权重： $OPT(\mathrm{j}-1)$
- 状态转移方程：$OPT(j)=\max\{OPT(j-1),OPT(p(j))+w_j\}$
- 边界条件： $OPT(0)=0$

伪代码

```
OPT[0] ← 0   // 基础情况，0 个任务的最大权重为 0
S[j][2] ←{}      // 备忘录S，记录选择路径，初始化为空
For j ← 1 to n do:
    If OPT[p(j)] + w[j] ≥ OPT[j-1] then
        OPT[j] ← OPT[p(j)] + w[j]   // 选择任务 j
        S[j][0] ← true              // 标记任务 j 被选中
        S[j][1] ← p(j)              // 记录前一个选中的任务 p(j)
    Else
        OPT[j] ← OPT[j-1]  // 不选择任务 j，继承上一个最优解
        S[j][0] ← false             // 标记任务 j 未被选中
        S[j][1] ← j-1               // 记录从上一个任务继承结果
     Endif
  Endfor
```

解追踪

```
Traceback(j):
    If j = 0 then return // 递归的终止条件
    If S[j][0] = true then
        输出任务 j
        Traceback(S[j][1]) // 继续追溯到前一个选中的任务
    Else
        Traceback(S[j][1]) // j 未被选中，追溯到 j-1
```

任务编号 j	$OPT(j)$	$S[j][0]$ （是否选中）	$S[j][1]$ （上一个选中的任务）
1	7	true	0
2	7	false	1
3	8	true	0
4	16	true	1
5	16	false	4
6	23	true	4

算法复杂度

- 根据状态转移方程

$$OPT(j) = max\{OPT(j-1), OPT(p(j)) + w_j\}$$

- 计算 $p(j)$：先排序，再二分法查找，$O(n \log n)$
  - 计算开销为： $O(n \log n) + O(n) = O(n \log n)$
- 追踪解的开销： $O(n)$
- 总开销为： $O(n \log n)$

问题4：投资问题

- 问题：m元钱，n项投资，$f_i(x)$将x元投入第i个项目的收益。求使得总收益最大的投资方案。

- 建模：问题的解是向量$x =< x_1, x_2, \dots x_n >$， x_i是投给项目i的钱数，$i = 1,2, \dots, n$
- 目标函数：$\max\{f_1(x_1) + f_2(x_2) + \cdots + f_n(x_n)\}$
- 约束条件：$x_1 + x_2 + \cdots + x_n = m, x_i \in \mathrm{N}$

例子

- 5万元，4个项目，收益函数如下图所示

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

动态规划求解

- 状态由参数k和x界定
  - k: 考虑对项目 $1,2,\ldots,k$ 的投资
  - x: 投资总钱数
- 状态定义
  - $k=1,2,\ldots,n$，对于给定的k，$x=1,2,\ldots,m$
  - $F_k(x)$：x万元钱投给前k个项目的最大效益
  - x_k：投到第k个项目的钱数
- 确定 x 元钱对前 k 个项目的分配方案的状态转移方程和边界条件如下:

$$F_k(x)=\max_{0\le x_k\le x}\{f_k(x_k)+F_{k-1}(x-x_k)\},k>1$$

$$F_1(x)=f_1(x)$$

k=1 计算

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

$$k = 1, F_1(1) = 11, F_1(2) = 12, F_1(3) = 13, F_1(4) = 14, F_1(5) = 15$$

k=2 计算

- 投入1的分配方案：(0,1), (1,0)
  - $F_2(1) = \max\{\boldsymbol{f_2(1)}, f_1(1)\} = 11$
- 投入2的分配方案：(0,2), (2,0), (1,1)
  - $F_2(2) = \max\{f_2(2), F_1(2), F_1(1) + f_2(1)\} = 12$
- 分配方案：(0,3), (3,0), (2,1), (1,2)
  - $F_2(3) = \max\{f_2(3), F_1(3), F_1(2) + f_2(1), F_1(1) + f_2(2)\} = 16$
- 继续计算： $F_2(4) = 21, F_2(5) = 26$

x	$f_1(x)$	$f_2(x)$
0	0	0
1	11	0
2	12	5
3	13	10
4	14	15
5	15	20

伪代码

```
1.  for x ← 0 to m do
2.      F[1][x] ← f[1][x]
3.      s[1][x] ← x
4.  for k ← 2 to n do
5.      for x ← 0 to m do
6.          F[k][x] ← -∞
7.          for xk ← 0 to x do
8.              value ← f[k][xk]+ F[k-1][x-xk]
9.              if value > F[k][x] then
10.                 F[k][x] ← value
11.                 s[k][x] ← xk
```

```
12. // 回溯最优投资方案
13. x ← m

14. for k ← n downto 1 do
15.     invest[k] ← s[k][x]
16.     x ← x - invest[k]
```

DP表

例： 5 万元、4 个项目 —— DP 表与解追踪表

1. 收益函数 $f_k(x)$

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

2. DP 表：最大收益 $F_k(x)$

k \ x	0	1	2	3	4	5
1	0	11	12	13	14	15
2	0	11	16	21	27	32
3	0	11	13	30	42	72
4	0	20	31	33	50	**61**

3. 解追踪表：最优投资额 $s_k(x)$

k \ x	0	1	2	3	4	5
1	0	1	2	3	4	5
2	0	0	0	2	3	4
3	0	0	1	3	3	4
4	0	1	1	1	1	**1**

4. 状态转移方程与边界条件

状态转移方程：$$F_k(x) = \max_{0 \le x_k \le x} \left\{ f_k(x_k) + F_{k-1}(x - x_k) \right\}$$

边界条件：$F_1(x) = f_1(x)$

$s_k(x)$ 记录使 $F_k(x)$ 最大时，第 k 个项目的投资额 x_k。

5. 从 $(k, x) = (4, 5)$ 开始回溯

步骤	当前状态 (k, x)	$s_k(x)$	第 k 个项目投资	剩余资金	该项目收益
1	$(4, 5)$	1	$x_4 = 1$	4	$f_4(1) = 20$
2	$(3, 4)$	3	$x_3 = 3$	1	$f_3(3) = 30$
3	$(2, 1)$	0	$x_2 = 0$	1	$f_2(0) = 0$
4	$(1, 1)$	1	$x_1 = 1$	0	$f_1(1) = 11$

核对：$F_2(2) = \max\{12, 11, 5\} = 12$

6. 最优投资方案

$x_1 = 1,\ x_2 = 0,\ x_3 = 3,\ x_4 = 1$

总投资：$1 + 0 + 3 + 1 = 5$ （满足约束）

各项目收益：

- 项目 1：11
- 项目 2：0
- 项目 3：30
- 项目 4：20

最大总收益：11 + 0 + 30 + 20 = 61

复杂度分析

- m 元 n 个投资任务（ $1 \le k \le n, 1 \le x \le m$) ）
  - $F_k(x) = \max\limits_{0 \le x_k \le x} \{f_k(x_k) + F_{k-1}(x - x_k)\}, k > 1$
  - $F_1(x) = f_1(x)$

- x_k 有 $x+1$种可能，计算$F_k(x)(2 \le k \le n, 1 \le x \le m)$需要$x+1$次加法，$x$次比较

$$\sum_{k=2}^{n}\sum_{x=1}^{m} x + 1 = \frac{(n-1)m(m+3)}{2}$$

$$\sum_{k=2}^{n}\sum_{x=1}^{m} x = \frac{(n-1)m(m+1)}{2}$$

$$O(nm^2)$$

背包问题

- **问题：**一个旅行者准备随身携带一个背包。可以放入背包的物品有 n 种, 每种物品的重量和价值分别为w_j, v_j. 如果背包的最大重量限制是 b, 怎样选择放入背包的物品以使得背包的价值最大? 假设 w_j, v_j, b都是整数

- **建模：**解是向量$x =< x_1, x_2, \ldots x_n >$， x_i是装入背包的第i种物品的个数
- **目标函数：** $\max \sum_{i=1}^{n} v_i x_i$
- **约束条件：** $\sum_{i=1}^{n} w_i x_i \leq b, x_i \in N$

状态定义

- 状态由参数k和 y 界定：
  - k：只考虑前 k 种物品；
  - y：当前允许使用的背包容量，$0 \le y \le b$。
- $F_k(y)$：装前k种物品，总重量不超过y，背包的最大价值
- 定义

$$\boxed{F_k(y) = \text{只选择前 } k \text{ 种物品，在总重量不超过 } y \text{ 时能够获得的最大价值}}$$

状态转移

- 方程一： $F_k(y) = \max\{F_{k-1}(y), F_k(y - w_k) + v_k\}$
- 方程二：$F_k(y) = \max_{0 \le x_k \le \left\lfloor \frac{y}{w_k} \right\rfloor} \{x_k v_k + F_{k-1}(y - x_k w_k)\}$
- 边界条件：
  - $F_0(y) = 0, 0 \le y \le b, F_k(0) = 0, 0 \le k \le n$
  - $F_1(y) = \left\lfloor \frac{y}{w_1} \right\rfloor v_1, F_k(y) = -\infty \ (y < 0, \text{超重})$

等价

- $F_k(y) = \max\{F_{k-1}(y), F_k(y - w_k) + v_k\}$
- 含义：要么不用第 k 项；要么再拿 1 件第 k 项，然后在容量 $y - w_k$上继续做“前 k 项完全背包”。

- $F_k(y) = \max\limits_{0 \le x_k \le \left\lfloor \frac{y}{w_k} \right\rfloor} \{x_k v_k + F_{k-1}(y - x_k w_k)\}$
- 含义：枚举第 k 项取 x_k 件，其余容量给前 $k - 1$ 项。

- 哪个复杂度更低？

解追踪：标记函数

- $i_k(y)$：装入前 k 种物品，总重不超过 y，背包达最大价值时装入物品的最大标号
- **也就是：**最优解中被选择物品的最大编号

$$i_k(y)=\begin{cases} i_{k-1}(y) & F_{k-1}(y)>F_k(y-w_k)+v_k \\ k & F_{k-1}(y)\le F_k(y-w_k)+v_k \end{cases}$$

$$i_1(y)=\begin{cases} 0 & y<w_1 \\ 1 & y\ge w_1 \end{cases}$$

DP表

$v_1 = 1,\ \ v_2 = 3,\ \ v_3 = 5,\ v_4 = 9,$

$w_1 = 2,\ \ w_2 = 3,\ w_3 = 4,\ w_4 = 7,$

$b = 10$

$F_k(y)$ 的计算表如下：

k \ y	1	2	3	4	5	6	7	8	9	10
1	0	1	1	2	2	3	3	4	4	5
2	0	1	3	3	4	6	6	7	9	9
3	0	1	3	5	5	6	8	10	10	11
4	0	1	3	5	5	6	9	10	10	12

前k个物品
k=?

背包重量

解追踪

k \ y	1	2	3	4	5	6	7	8	9	10
1	0	1	1	1	1	1	1	1	1	1
2	0	1	2	2	2	2	2	2	2	2
3	0	1	2	3	3	3	3	3	3	3
4	0	1	2	3	3	3	4	3	4	4

- $i_4(10) = 4 \Rightarrow x_4 \geq 1 \Rightarrow i_4(10 - w_4) = i_4(3) = 2$
- $i_4(3) = 2 \Rightarrow x_2 \geq 1, x_4 = 0, x_3 = 0$
- $i_2(3 - w_3) = i_2(0) = 0 \Rightarrow x_2 = 1, x_1 = 0$

时间复杂度分析

- DP 表格 n 行 b列（n个物品，重量上限b）

$$F_k(y) = \max\{F_{k-1}(y), F_k(y - w_k) + v_k\}$$

- 计算 $F_k(y)$ 只需要 常数时间
- 时间复杂度：$O(nb)$

- 伪多项式时间算法：b是整数，不是输入长度，因此，算法的输入规模是 n和$\log b$

背包问题的推广

- 物品受限背包：第 i 个物品最多装入 n_i 个
- 0-1背包问题：第 i 个物品最多装入1个
- 多背包问题：m 个背包，第 j 个背包最多装入重量为$B_j, j = 1,2, \dots, m$
- 二维背包问题：每个物品有重量$w_{i=1,2,\dots,n}$和体积$t_{i=1,2,\dots,n}$，每个背包有重量限制b和体积限制v

最长公共子序列

- 设序列 X, Z
- $X =< x_1, x_2, \dots x_m >, Z =< z_1, z_2, \dots, z_k >$

- 若存在X的元素构成严格递增序列，使得$z_j = x_i, j = 1,2,3, \dots, k$, 则称$Z$是$X$的子序列

- X与Y的公共子序列Z：Z是X和Y的子序列
- 子序列长度：子序列元素个数

最长公共子序列

- 问题：给定序列$X =< x_1, x_2, \ldots x_m >, Y =< y_1, y_2, \ldots, y_n >$, 求$X$和$Y$的最长公共子序列

- 例子：

X: A B C B D A B

Y: B D C A B A

最长公共子序列: B C B A

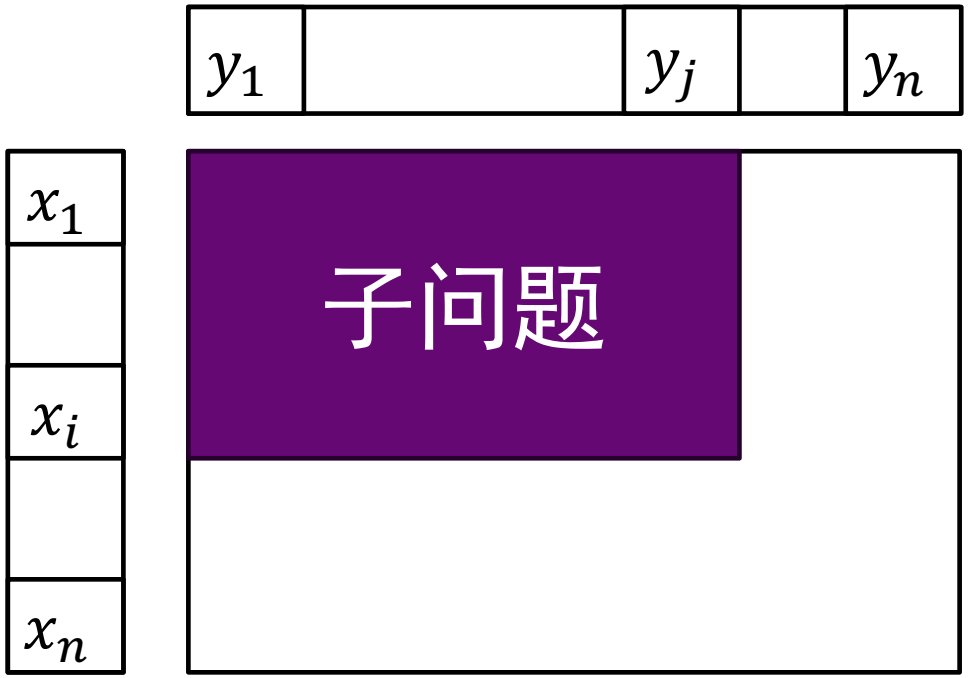
暴力穷举

- 设$m \leq n$，依次检查X的子序列在Y中是否出现

- 时间复杂度： $O(n2^m)$
- 每个子序列 $O(n)$ 时间，X有2^m个子序列

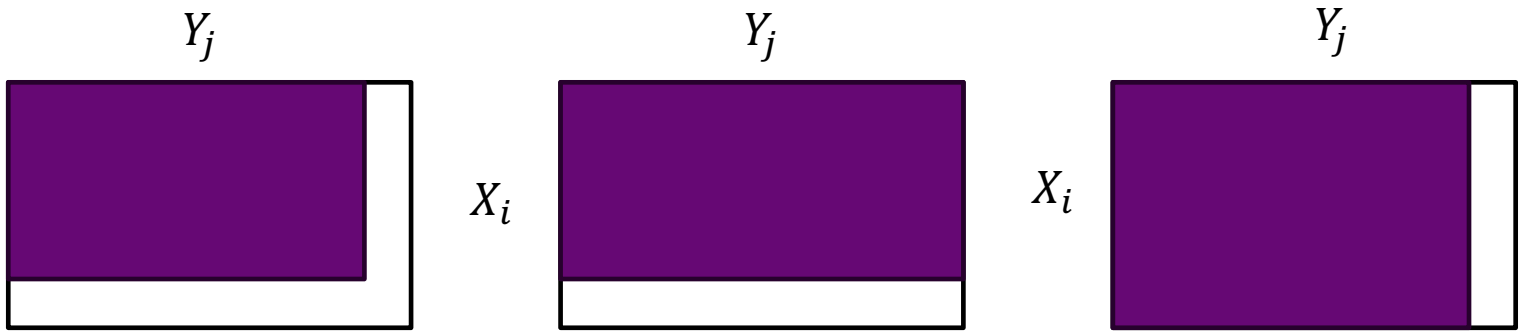
子问题/状态定义

- 用 参数 i 和 j 定义状态
- X的终止位置是i, Y的终止位置是j
- $X =< x_1, x_2, \ldots x_m >, Y =< y_1, y_2, \ldots, y_n >$



子问题的依赖关系

- 设$X_i =< x_1, x_2, \dots x_i >, Y_j =< y_1, y_2, \dots, y_j >$， $Z_j =< z_1, z_2, \dots, z_i >$
- 设$Z =< z_1, z_2, \dots, z_k >$为 X 和 Y 的最长公共子序列
  - 若$x_m = y_n \Rightarrow z_k = x_m = y_n \Rightarrow Z_{k-1}$ 是X_{m-1}和Y_{n-1}的最长公共子序列
  - 若$x_m \neq y_n, z_k \neq x_m \Rightarrow Z$是$X_{m-1}$和$Y$的最长公共子序列
  - 若$x_m \neq y_n, z_k \neq y_n \Rightarrow Z$是$X$和$Y_{n-1}$的最长公共子序列



状态转移方程

- 设$X_i =< x_1, x_2, \dots x_i >, Y_j =< y_1, y_2, \dots, y_j >$
- 设$C[i,j]$为X_i和Y_i的最长公共子序列的大小

$$C[i,j] = \begin{cases} 0 & \text{若} i = 0 \text{或} j = 0 \\ C[i-1, j-1] + 1 & \text{若} i, j > 0, x_i = y_i \\ \max\{C[i, j-1], C[i-1, j]\} & \text{若} i, j > 0, x_i \neq y_i \end{cases}$$

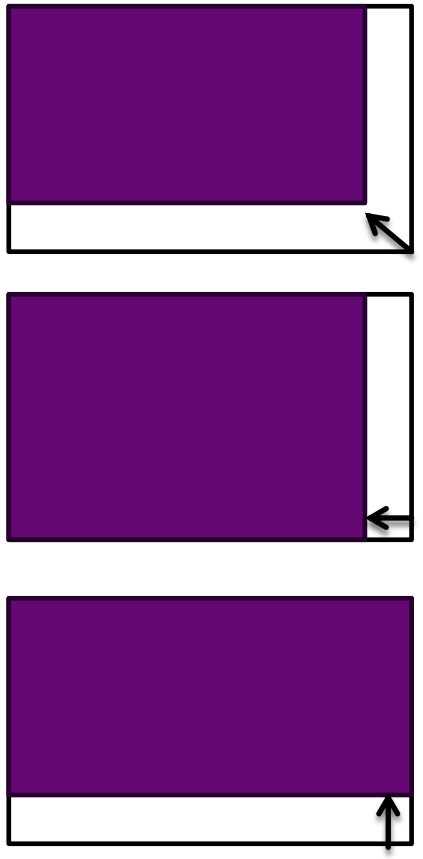
标记函数

- 标记函数$B[i,j]$, 值为↖,←,↑

$$C[i,j] = C[i-1,j-1] + 1$$

$$C[i,j] = C[i,j-1]$$

$$C[i,j] = C[i-1,j]$$



伪代码和时间复杂度

```
1.  for i←1 to m do          //行1-4边界情况
2.      C[i,0]←0
3.  for i←1 to n do
4.      C[0,i]←0
5.  for i←1 to m do
6.      for j←1 to n do
7.          if X[i]=Y[j]
8.          then C[i,j]←C[i–1,j–1]+1
9.               B[i,j]←'↖'
10.         else if C[i–1,j] ≥ C[i,j–1]
11.              then C[i,j]←C[i–1,j]
12.                   B[i,j]←'↑'
13.              else C[i,j]←C[i,j–1]
14.                   B[i,j]←'←'
```

动态规划算法：时间为$O(mn)$

解追踪：每一步至少 缩小 X 或 Y 的长度 1，时间$\Theta(m+n)$

空间：$\Theta(mn)$

例子

输入：X=<A,B,C,B,D,A,B>, Y=<B,D,C,A,B,A>,

标记函数：

	1	2	3	4	5	6
1	$B[1,1]=\uparrow$	$B[1,2]=\uparrow$	$B[1,3]=\uparrow$	$B[1,4]=\nwarrow$	$B[1,5]=\leftarrow$	$B[1,6]=\nwarrow$
2	$B[2,1]=\nwarrow$	$B[2,2]=\leftarrow$	$B[2,3]=\leftarrow$	$B[2,4]=\uparrow$	$B[2,5]=\nwarrow$	$B[2,6]=\leftarrow$
3	$B[3,1]=\uparrow$	$B[3,2]=\uparrow$	$B[3,3]=\nwarrow$	$B[3,4]=\leftarrow$	$B[3,5]=\uparrow$	$B[3,6]=\uparrow$
4	$B[4,1]=\uparrow$	$B[4,2]=\uparrow$	$B[4,3]=\uparrow$	$B[4,4]=\uparrow$	$B[4,5]=\nwarrow$	$B[4,6]=\leftarrow$
5	$B[5,1]=\uparrow$	$B[5,2]=\uparrow$	$B[5,3]=\uparrow$	$B[5,4]=\uparrow$	$B[5,5]=\uparrow$	$B[5,6]=\uparrow$
6	$B[6,1]=\uparrow$	$B[6,2]=\uparrow$	$B[6,3]=\uparrow$	$B[6,4]=\nwarrow$	$B[6,5]=\uparrow$	$B[6,6]=\nwarrow$
7	$B[7,1]=\uparrow$	$B[7,2]=\uparrow$	$B[7,3]=\uparrow$	$B[7,4]=\uparrow$	$B[7,5]=\uparrow$	$B[7,6]=\uparrow$

解为X[2], X[3],X[4], X[6], 即 B C B A

二分查找树

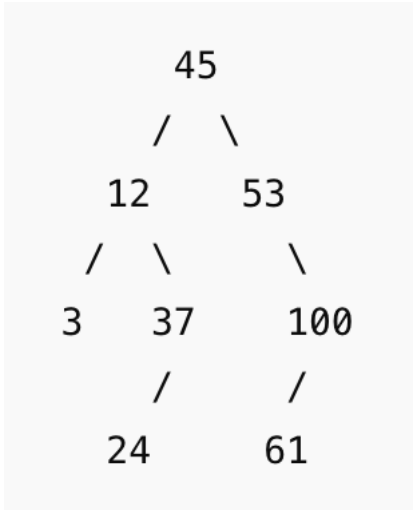
- 二分查找树（Binary Search Tree，常叫“二叉搜索树”，简称 BST）是一种**每个结点最多两个孩子**的树形数据结构，并满足**有序性不变式**：
- 对任一结点 x：左子树里所有键 < x.key；右子树里所有键 > x.key（或按约定处理相等）。
- 这个性质对整棵左/右子树**递归成立**。

构建二分查找树

- 假设有一组数：

$$[45, 12, 53, 3, 37, 100, 24, 61]$$

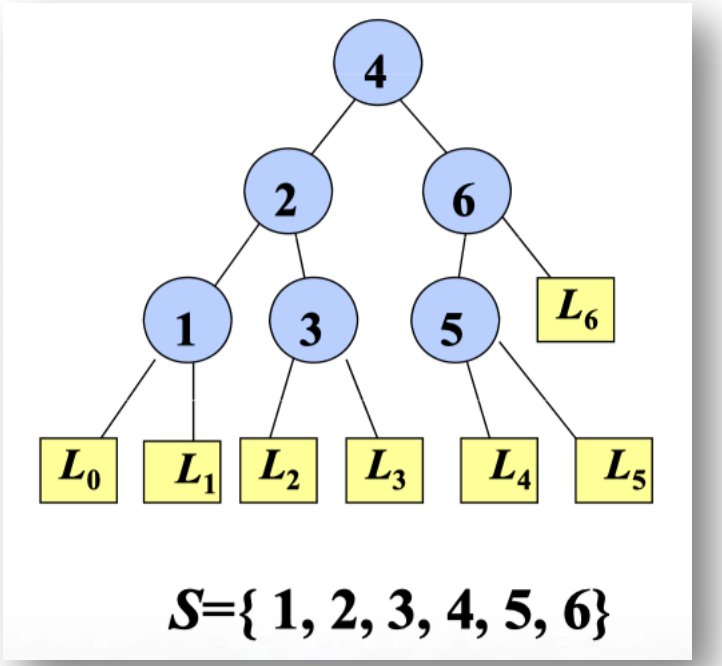
- 第一个数 45 → 根节点。
- 读入 12，比 45 小 → 插入左子树。
- 读入 53，比 45 大 → 插入右子树。
- 读入 3，比 45 小、比 12 小 → 插入到 12 的左子树。
- 读入 37，比 45 小、比 12 大 → 插入 12 的右子树。
- 依次类推……



- 对**有序数组**构建二叉查找树的最佳做法是：**每次取中点为根，左右区间递归建子树**。这样能得到**高度最小/近似平衡**的 BST，查找为 $O(\log n)$。

二分查找树

- 设集合S为排序的n个元素 $x_1 < x_2 < \cdots < x_n$，将这些元素存储在一棵二叉查找树的结点上
- 目的是查找 x是否在这些数中；如果x不在，确定x在哪个空隙（叶子结点）
- 检索方法：
  - 1. 初始，x与根元素比较；
  - 2. x<根元素，递归进入左子树；
  - 3. x>根元素，递归进入右子树；
  - 4. x=根元素，算法停止，输出x；
  - 5. x到达叶结点，算法停止，输出x不在数组中



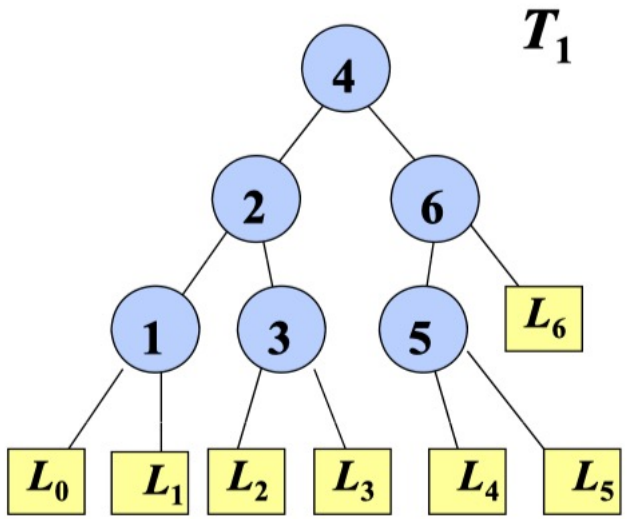
数据元素存取概率分布

- 空隙：$(x_0, x_1), (x_1, x_2), \ldots, (x_n, x_{n+1})$
  - $x_0 = -\infty, x_{n+1} = +\infty$
- 给定序列$S =< x_1, x_2, \ldots, x_n >$
  - $x = x_i$的概率为b_i
  - $x \in (x_i, x_{i+1})$的概率为a_i
- x的存取分布概率为

$$P =< a_0, b_1, a_1, b_2, \ldots, b_n, a_n >$$

例子

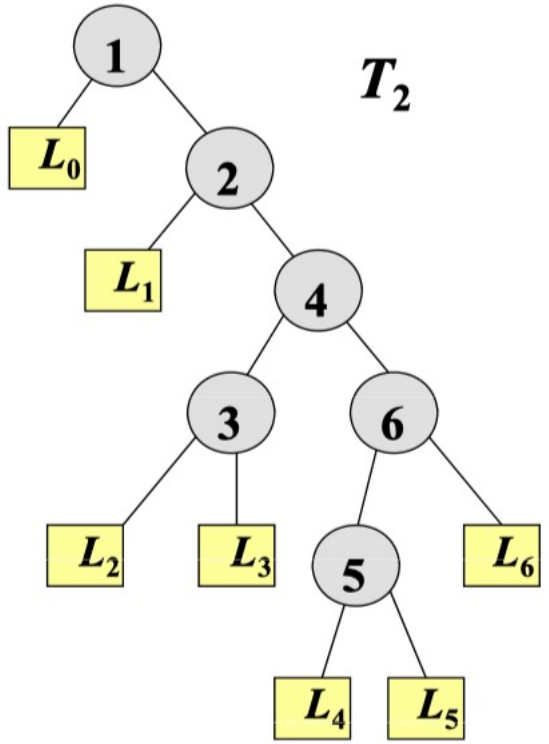
- $S =< 1, 2, 3, 4, 5, 6 >$
- $P =< 0.04, 0.1, 0.01, 0.2, 0.05, 0.2, 0.02, 0.1, 0.02, 0.1, 0.07, 0.05, 0.04 >$
- $m(T_1)$：在这棵 BST 上，按给定概率做一次查找，平均需要的比较次数?



$$m(T_1) = [1 * 0.1 + 2 * (0.2 + 0.05) + 3 * (0.1 + 0.2 + 0.1)]$$
$$+[3 * (0.04 + 0.01 + 0.05 + 0.02 + 0.02 + 0.07) + 2 * 0.04]$$
$$= 1.8 + 0.71 = 2.51$$

例子

- $S =< 1, 2, 3, 4, 5, 6 >$
- $P =< 0.04, 0.1, 0.01, 0.2, 0.05, 0.2, 0.02, 0.1, 0.02, 0.1, 0.07, 0.05, 0.04 >$



$$m(T_2) = [1 * 0.1 + 2 * 0.2 + 3 * 0.1 + 4 * (0.2 + 0.05) + 5 * 0.1]$$
$$+[1 * 0.04 + 2 * 0.01 + 4 * (0.05 + 0.02 + 0.04) + 5 * (0.02 + 0.07)]$$
$$= 2.3 + 0.95 = 3.25$$

最优二分查找树

- 给定数据集：$S =< x_1, x_2, \dots, x_n >$
- 给定概率分布：$P =< a_0, b_1, a_1, b_2, \dots, b_n, a_n >$
- 结点x_i在T中的深度是$d(x_i), i = 1,2, \dots, n$ （根的深度取为 0）
- 空隙L_j在T中的深度是$d(L_j), j = 0,1,2, \dots, n$
- 平均比较次数为：

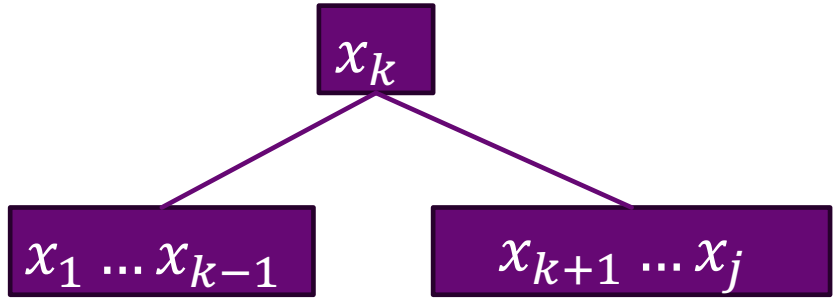
找到还要做一次比较

$$t = \sum_{i=1}^{n} b_i\big(1 + d(x_i)\big) + \sum_{i=0}^{n} a_i d(L_i)$$

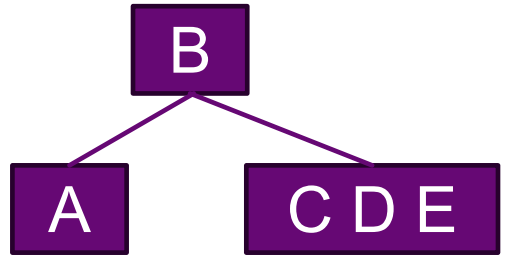
- 问题：给定数据集 S 和相关存取概率分布 P，求一棵最优的 (即平均比较次数最少的) 二分检索树

状态定义

- 子问题由边界(i,j)决定
- 数据集$S[i,j] =< x_i, x_{i+1}, \dots, x_j >$
- 概率分布$P[i,j] =< a_{i-1}, b_i, a_i, b_{i+1}, \dots, b_j, a_j >$
- 可以以 x_k 作为根，划分成两个子问题：
  - $S[i,k-1], P[i,k-1]$
  - $S[k+1,j], P[k+1,j]$



- 例子： $S =< A, B, C, D, E >, P =< 0.04, 0.1, 0.02, 0.3, 0.02, 0.1, 0.05, 0.2, 0.06, 0.1, 0.01 >$
  - 以B为根
  - $S[1,1] =< A >$ ， $P[1,1] =< 0.04, 0.1, 0.02 >$
  - $S[3,5] =< C, D, E >, P[3,5] =< 0.02, 0.1, 0.05, 0.2, 0.06, 0.1, 0.01 >$



- 令$w[i,j]$ 为 $P[i,j]$ 中所有概率之和：$w[i,j] = \sum_{p=i-1}^{j} a_p + \sum_{q=i}^{j} b_q$

状态定义、状态转移方程、边界条件

- 设 $m[i,j]$ 是相对于输入 $S[i,j]$ 和 $P[i,j]$ 的最优二叉搜索树的平均比较次数（状态）
- 状态转移方程为：

$$m[i,j] = \min_{i \le k \le j}\{m[i,k-1] + m[k+1,j] + w[i,j]\}\,(1 \le i \le j \le n)$$

- 边界条件：
  - $m[i,i-1] = 0, i = 1,2,\ldots,n$(子树为空的情况)

假设 $S = \{x_1, x_2, x_3\}$

当 $k = 1$ 作为根时：

- 左子树区间：$[1,0]$（空）
- 右子树区间：$[2,3]$

→ 左子树为空，对应 $m[1,0] = 0$

当 $k = 3$ 作为根时：

- 左子树区间：$[1,2]$
- 右子树区间：$[4,3]$（空）

→ 右子树为空，对应 $m[4,3] = 0$

状态转移方程为什么这样定义?

- $m[i,j]_k$：根为 x_k 时的二分检索树平均比较次数的最小值
- 选 k为根后，左子树 $[i..k-1]$和右子树 $[k+1..j]$的每个概率（成功或失败）深度都 +1，所以要把它们各自的概率和 $w[\mathrm{i},\mathrm{k}-1]$、$w[\mathrm{k}+1,\mathrm{j}]$ 加上；
- 根键 k自己要比较 1 次，增加 $1\times b_k$。

$$
\begin{aligned}
&m[i,j]_k\\
&=(m[i,k-1]+w[i,k-1])+(m[k+1,j]+w[k+1,j])+1\times b_k\\
&=(m[i,k-1]+m[k+1,j])+(w[i,k-1]+b_k+w[k+1,j])\\
&=(m[i,k-1]+m[k+1,j])+(\sum_{p=i-1}^{k-1}a_p+\sum_{q=i}^{k-1}b_q)+b_k+(\sum_{p=k}^{j}a_p+\sum_{q=k+1}^{j}b_q)\\
&=(m[i,k-1]+m[k+1,j])+\sum_{p=i-1}^{j}a_p+\sum_{q=i}^{j}b_q\\
&=m[i,k-1]+m[k+1,j]+w[i,j]
\end{aligned}
$$

- 平均比较次数最小值：在所有 k 的情况下 $m[i,j]_k$的最小值
- $m[i,j]=\min\{m[i,j]_k \mid i\le k\le j\}$

伪代码

```
INF = +∞
# m[i][j]：仅用键 i..j 的最小期望代价（空区间为 0）
# root[i][j]：使 m[i][j] 取最小值的根 r
for i = 1..n+1:
  m[i][i-1] = 0                    # 基例：空区间

for len = 1..n:                    # 区间长度
  for i = 1..n-len+1:
    j = i + len - 1
    m[i][j] = INF
    for r = i..j:                  # 选择根 r
      cost = m[i][r-1] + m[r+1][j] + W(i, j)
      if cost < m[i][j]:
        m[i][j] = cost
        root[i][j] = r

return m[1][n], root               # 最小期望代价与根表（可恢复树）
```

例子：3个关键字的最优二叉查找树

元素	成功概率 (b_i)	失败概率 (a_i)（前后空隙）
(x_1)	0.3	$(a_0 = 0.1,\ a_1 = 0.1)$
(x_2)	0.2	$(a_2 = 0.1)$
(x_3)	0.1	$(a_3 = 0.1)$

步骤 1：计算各区间概率和 $w[i,j]$

$$
\begin{aligned}
w[1,1] &= a_0 + b_1 + a_1 = 0.5 \\
w[2,2] &= a_1 + b_2 + a_2 = 0.4 \\
w[3,3] &= a_2 + b_3 + a_3 = 0.3 \\
w[1,2] &= a_0 + a_1 + a_2 + b_1 + b_2 = 0.8 \\
w[2,3] &= a_1 + a_2 + a_3 + b_2 + b_3 = 0.6 \\
w[1,3] &= a_0 + a_1 + a_2 + a_3 + b_1 + b_2 + b_3 = 1.0
\end{aligned}
$$

步骤 2：区间长度为 1 时

$$m[1,1] = 0 + 0 + w[1,1] = 0.5, m[2,2] = 0.4, m[3,3] = 0.3$$

例子：3个关键字的最优二叉查找树

区间 [1,2]：

根(k)	左子树	右子树	加权	总代价
(x_1)	$(m[1,0]=0)$	$(m[2,2]=0.4)$	$(w[1,2]=0.8)$	$\mathbf{1.2}$
(x_2)	$(m[1,1]=0.5)$	$(m[3,2]=0)$	$(w[1,2]=0.8)$	1.3

区间 [2,3]：

根(k)	左子树	右子树	加权	总代价
(x_2)	$(m[2,1]=0)$	$(m[3,3]=0.3)$	$(w[2,3]=0.6)$	$\mathbf{0.9}$
(x_3)	$(m[2,2]=0.4)$	$(m[4,3]=0)$	$(w[2,3]=0.6)$	1.0

区间 [1,3]：

根(k)	左子树	右子树	加权	总代价
(x_1)	0	0.9	1.0	1.9
(x_2)	0.5	0.3	1.0	$\mathbf{1.8}$
(x_3)	1.2	0	1.0	2.2

动态规划在大模型的应用

- https://infini-ai-lab.github.io/Sequoia-Page/



尽管 推测解码（speculative decoding）近期已成为加速推理的一种有前景的方法，但现有方法在扩展到更大的推测预算，以及适应不同超参数和硬件平台方面，仍存在局限。

本文提出了 **Sequoia**，一种可扩展、稳健且具备硬件感知能力的推测解码算法。为提高可扩展性，Sequoia 引入了一种动态规划算法，为推测生成的 token 寻找最优树结构。为在不同设置下保持稳健的推测解码性能，Sequoia 采用了一种新的采样与验证方法，在不同解码温度下均优于已有方法。此外，Sequoia 还引入了一个硬件感知的树结构优化器，能够针对给定的硬件平台，自动选择 token 树的规模和深度，从而最大化推测解码性能。

实验结果表明，在 A100 GPU 上，Sequoia 对 Llama2-7B、Llama2-13B 和 Vicuna-33B 的解码加速比分别最高达到 **4.04 倍、3.73 倍和 2.27 倍**。在 L40 GPU 上的模型卸载（offloading）场景中，Sequoia 可将 Llama2-70B 的精确推理延迟降至最低 **0.56 秒/token**；相较于我们优化后的卸载系统（5.6 秒/token）、DeepSpeed-Zero-Inference 和 Hugging Face Accelerate，分别实现了 **9.96 倍、9.7 倍和 19.5 倍**的加速。

论文的目标就是：对于给定树大小，找到使期望接受 token 数最大的树结构

练习题

- 给定正整数 n 和非负整数 $K \geq 0$，以及一个长度为 n 的正整数数组$a_1, a_2, \ldots, a_n$。选择一个非空子序列（保持原相对顺序，但不要求连续），设被选中的元素下标依次为 $i_1 < i_2 < \cdots < i_m$。定义该子序列的代价为$\sum_{t=2}^{m} \left|a_{i_t} - a_{i_{t-1}}\right|$（若 $m = 1$，代价为 0）。请设计一个算法，统计代价恰好等于 K 的非空子序列个数，并给出算法的时间复杂度，要求不高于 $O(n^2 K)$，并分析空间复杂度。

参考答案

- 状态定义
  - 定义$\boxed{dp[i][k]}$，表示：**以 a_i作为最后一个元素，总代价恰好为 k 的非空子序列个数。**
  - 目标：$\boxed{\text{Ans} = \sum_{i=1}^{n} dp[i][K]}$
- 状态转移方程：
  - 对所有 $j < i$，令$d = |a_i - a_j|$，若 $d \le k$，则：
  - $\boxed{dp[i][k] \ += \ dp[j][k-d]}$
- 边界条件（初始化）：
  - 单独选一个元素时，代价为 0：$\boxed{dp[i][0] = 1}$，其余初始为：$\boxed{dp[i][k] = 0, k > 0}$，然后按照 $i = 1,2,\dots,n$依次计算。
- 复杂度分析
  - 共有$n(K+1) = O(nK)$个状态。
  - 对每个 i，枚举所有 $j < i$，再枚举代价 k：$\boxed{O(n^2K)}$
  - 空间保存整个 DP 表：$\boxed{O(nK)}$